



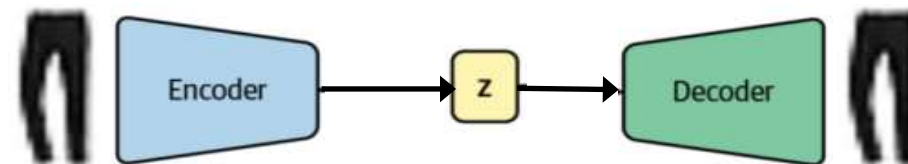
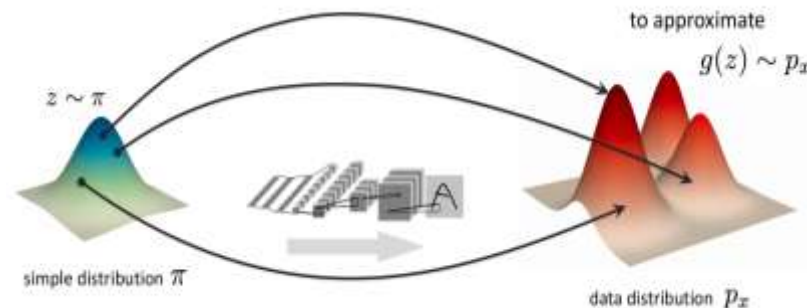
南京航空航天大学
NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

人工智能内容生成 (AIGC) 技术与应用

回顾

回顾

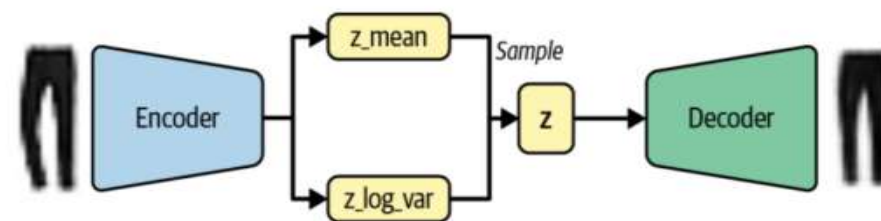
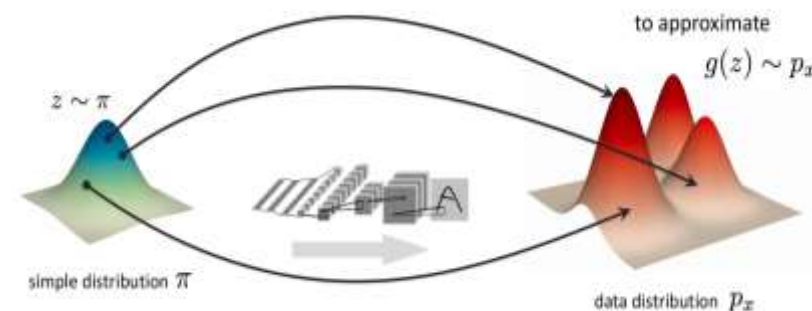
- 深度生成式模型 $p(x)$:
 - 简单分布采样 + 复杂映射 = 复杂分布采样
- 自编码器AE
 - 编码器：从高维输入到低维隐变量 z
 - 解码器：从低维隐变量 z 重构输入样本 x
- AE实现 z 到 x 的复杂映射，但隐变量 z 的数据分布并不好，无法实现采样生成



回顾

回顾

- **变分自编码器进一步约束隐变量 z 的分布与高斯分布相似,实现简单分布**
 - 编码器：将高维输入映射到低维隐变量 z 对应的高斯分布 $N(\mu, \sigma^2)$
 - 隐变量采样： $z \sim N(\mu, \sigma^2)$
 - 解码器：从低维隐变量 z 重构输入样本 x



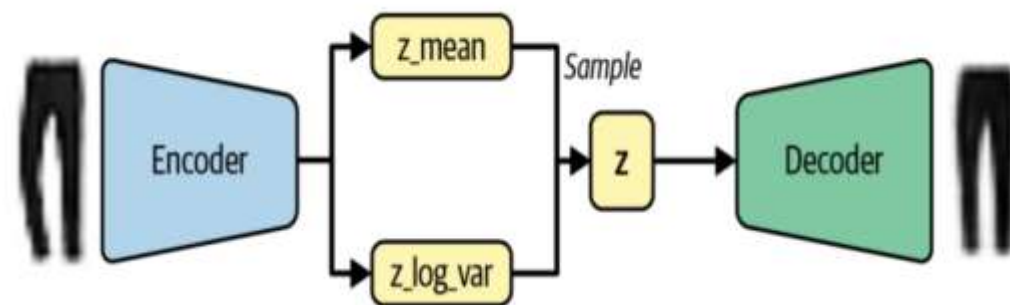
$$\mathcal{L} = \underbrace{\Sigma_x \{|x - x'|}_{\text{复杂映射}} + \underbrace{KL(N(\mu_x, \sigma_x^2) | N(0, I))}_{\text{简单分布}}$$

复杂映射

简单分布

VAE存在的问题

- 问题：重构损失默认数据各维度相互独立，但高维数据的元素间存在强关联性。
- 结果：生成的图像模糊，细节丰富度不足。



$$\mathcal{L} = \Sigma_x \{ |x - x'| + KL(N(\mu_x, \sigma_x^2) | N(0, 1)) \}$$

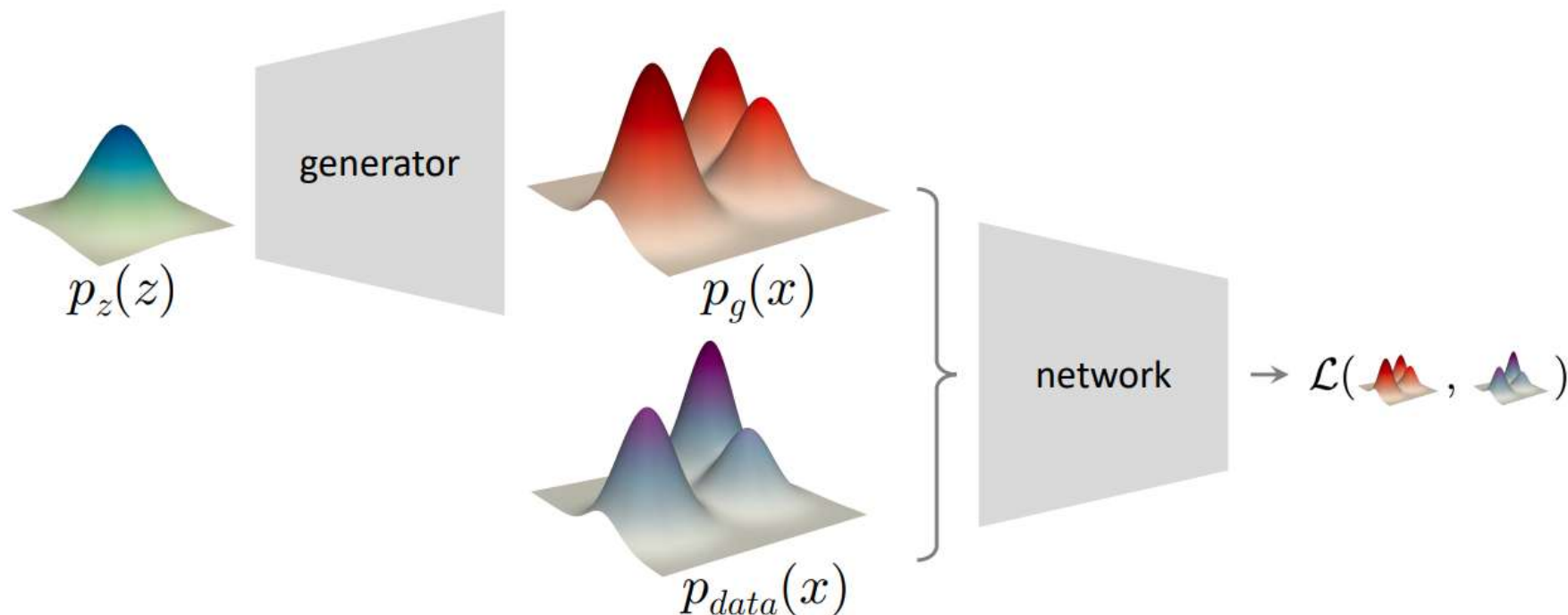
如何能够绕开像素独立性依赖构建z-x的映射
，生成高逼真、细节丰富的图像？



第四章

生成对抗网络 (GAN)

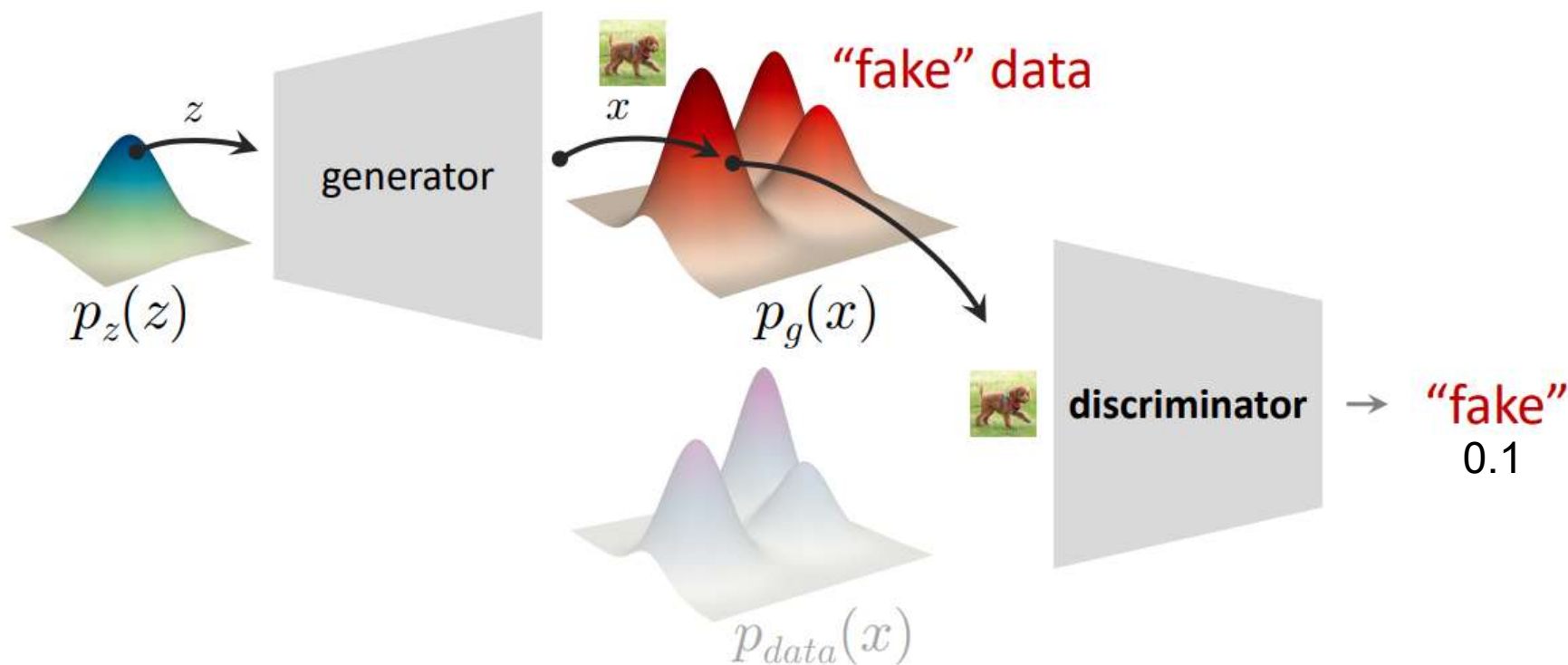
对抗神经网络 (GAN) --使用神经网络判断生成样本的真实性



使用神经网络，综合各像素之间的耦合信息，
判断样本的真实性

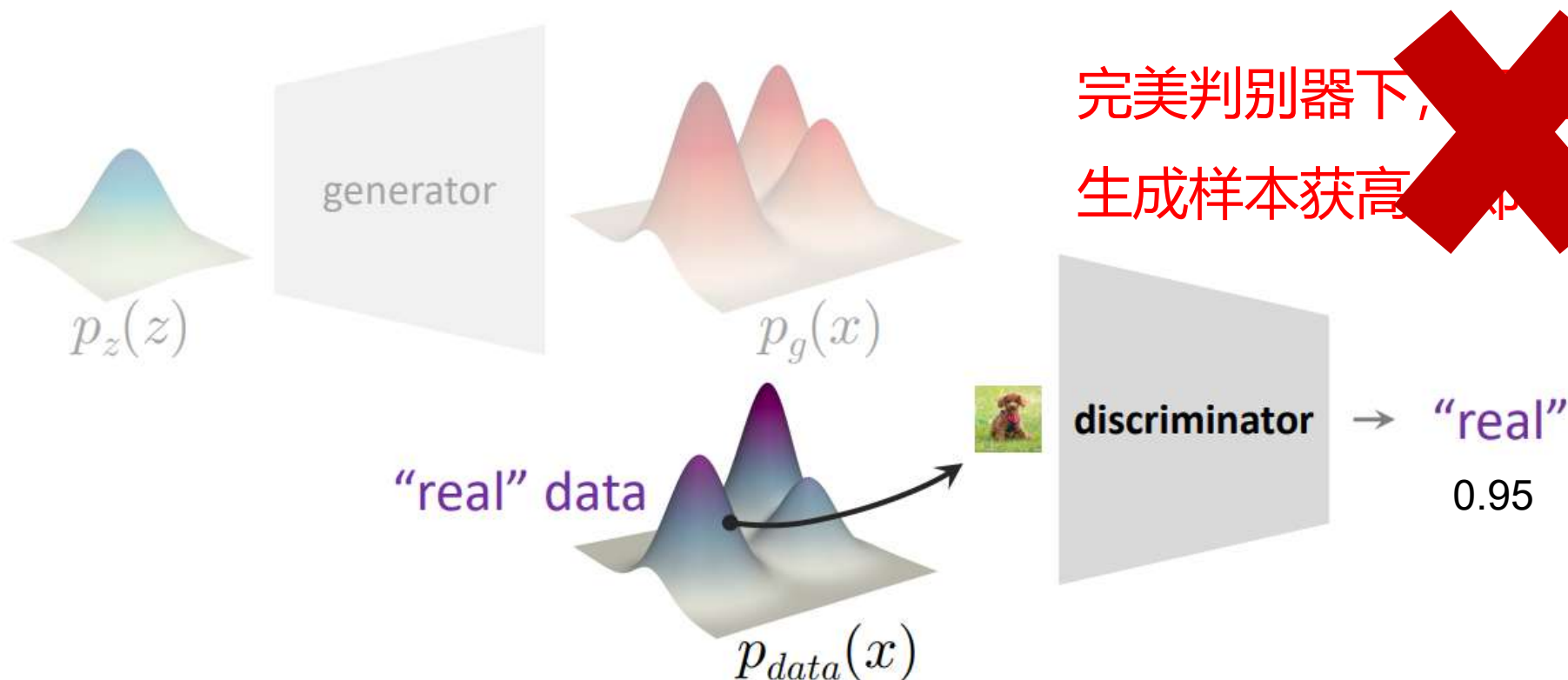
GAN--使用神经网络判断生成样本的真实性

- 生成器：从简单随机噪声中生成样本
- 判别器：判断输入样本是否是真实数据



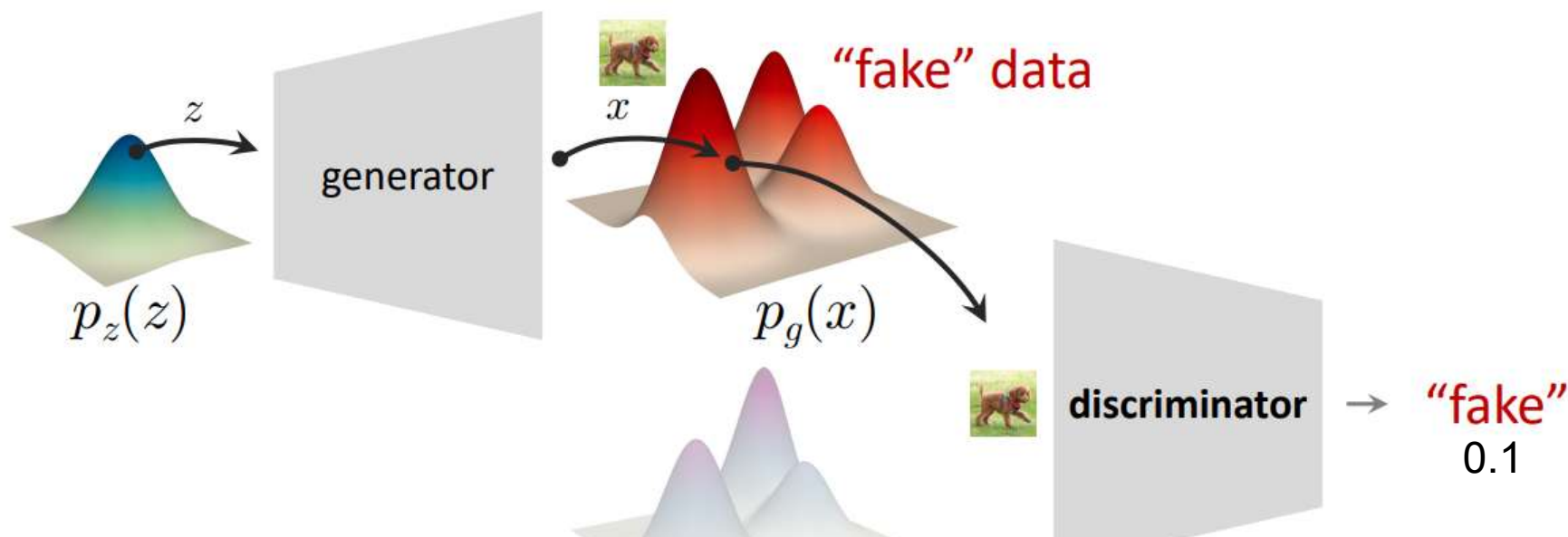
GAN--使用神经网络判断生成样本的真实性

- 生成器：从简单随机噪声中生成样本
- 判别器：判断输入样本是否是真实数据



GAN--使用神经网络判断生成样本的真实性

- 生成器：从简单随机噪声中生成样本
- 判别器：判断输入样本是否是真实数据



如何在优化生成器的同时
确保判别结果的合理性？

目录

- 第一节 对抗学习优化/GAN
- 第二节 理论探索
- 第三节 WGAN

第一节 对抗学习优化

对抗学习 (Adversarial Learning) — 伪造者与质检员



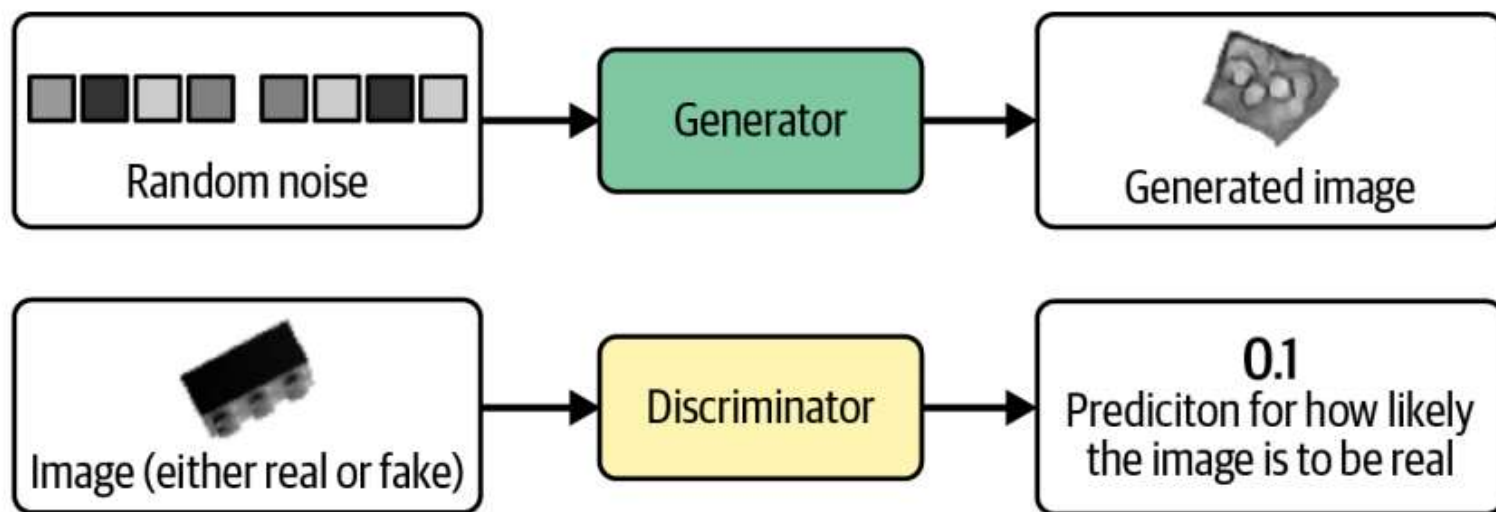
通过伪造者与质检员之间的“良性对抗”，不断提升“伪造”质量

角色映射 (左右分栏布局)	
质量控制员 (YOU)	伪造者 (FORGERS)
判别器(Discriminator)	生成器(Generator)
监听客户反馈 → 提升辨伪能力	初始技术: 随机噪声造废品
目标: 拦截所有假砖	对抗响应: 迭代优化伪造术

GAN (Generative Adversarial Network) --以砖块数据集为例

➤ 生成器（伪造者）与判别器（质检员）的**双网络对抗优化**框架。

- 生成器：从随机噪声生成砖块图像。
- 判别器：判断输入的砖块图像属于真实样本的概率。

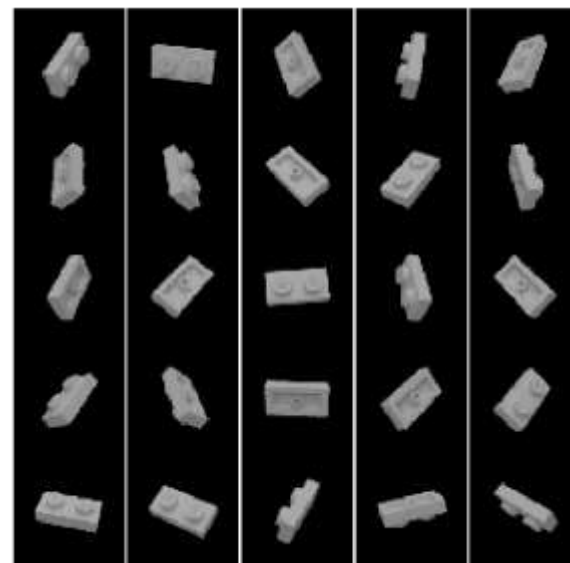


GAN (Generative Adversarial Network) --以砖块数据集为例

```
train_data = utils.image_dataset_from_directory(  
    "/app/data/lego-brick-images/dataset/",  
    labels=None,  
    color_mode="grayscale",  
    image_size=(64, 64),  
    batch_size=128,  
    shuffle=True,  
    seed=42,  
    interpolation="bilinear",  
)  
  
def preprocess(img):  
    img = (tf.cast(img, "float32") - 127.5) / 127.5  
    return img  
  
train = train_data.map(lambda x: preprocess(x))
```

➤ 砖块数据集:

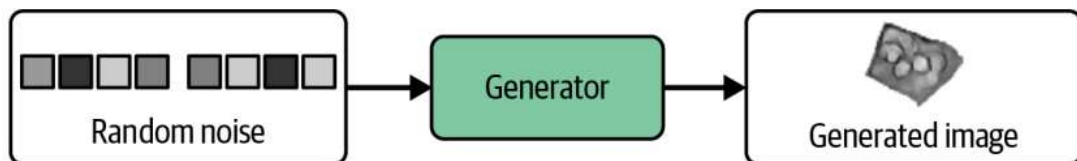
40,000个50种不同的砖
块图像 64*64分辨率



对抗学习优化

生成器

生成器： 输入是100维度的标准高斯噪声，输出为64*64的图像



```
generator_input = layers.Input(shape=(100,)) ❶
x = layers.Reshape((1, 1, 100))(generator_input) ❷
x = layers.Conv2DTranspose(
    512, kernel_size=4, strides=1, padding="valid", use_bias = False
)(x) ❸
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Conv2DTranspose(
    256, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Conv2DTranspose(
    128, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Conv2DTranspose(
    64, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
generator_output = layers.Conv2DTranspose(
    1,
    kernel_size=4,
    strides=2,
    padding="same",
    use_bias = False,
    activation = 'tanh'
)(x) ❹
generator = models.Model(generator_input, generator_output) ❺
```

对抗学习优化

生成器

Layer (type)	Output shape	Param #
InputLayer	(None, 100)	0
Reshape	(None, 1, 1, 100)	0
Conv2DTranspose	(None, 4, 4, 512)	819,200
BatchNormalization	(None, 4, 4, 512)	2,048
ReLU	(None, 4, 4, 512)	0
Conv2DTranspose	(None, 8, 8, 256)	2,097,152
BatchNormalization	(None, 8, 8, 256)	1,024
ReLU	(None, 8, 8, 256)	0
Conv2DTranspose	(None, 16, 16, 128)	524,288
BatchNormalization	(None, 16, 16, 128)	512
ReLU	(None, 16, 16, 128)	0
Conv2DTranspose	(None, 32, 32, 64)	131,072
BatchNormalization	(None, 32, 32, 64)	256
ReLU	(None, 32, 32, 64)	0
Conv2DTranspose	(None, 64, 64, 1)	1,024

```
generator_input = layers.Input(shape=(100,)) ①
x = layers.Reshape((1, 1, 100))(generator_input) ②
x = layers.Conv2DTranspose(
    512, kernel_size=4, strides=1, padding="valid", use_bias = False
)(x) ③
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Conv2DTranspose(
    256, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Conv2DTranspose(
    128, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Conv2DTranspose(
    64, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum=0.9)(x)
x = layers.LeakyReLU(0.2)(x)
generator_output = layers.Conv2DTranspose(
    1,
    kernel_size=4,
    strides=2,
    padding="same",
    use_bias = False,
    activation = 'tanh'
)(x) ④
generator = models.Model(generator_input, generator_output) ⑤
```

对抗学习优化

判别器

判别器：输入为64*64图像，输出为判断为真实图像的1维概率值



```
discriminator_input = layers.Input(shape=(64, 64, 1)) ❶
x = layers.Conv2D(64, kernel_size=4, strides=2, padding="same", use_bias = False)(discriminator_input) ❷
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(128, kernel_size=4, strides=2, padding="same", use_bias = False)(x)
x = layers.BatchNormalization(momentum = 0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(256, kernel_size=4, strides=2, padding="same", use_bias = False)(x)
x = layers.BatchNormalization(momentum = 0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(512, kernel_size=4, strides=2, padding="same", use_bias = False)(x)
x = layers.BatchNormalization(momentum = 0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(1, kernel_size=4, strides=1, padding="valid", use_bias = False, activation = 'sigmoid')(x)
discriminator_output = layers.Flatten()(x) ❸
```

对抗学习优化

判别器

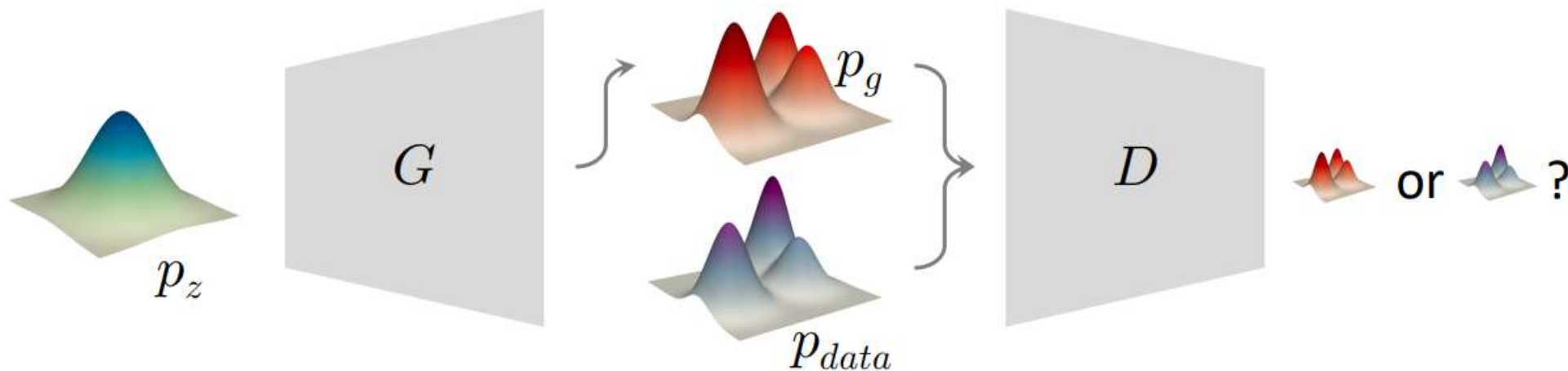
Layer (type)	Output shape	Param #
InputLayer	(None, 64, 64, 1)	0
Conv2D	(None, 32, 32, 64)	1,024
LeakyReLU	(None, 32, 32, 64)	0
Dropout	(None, 32, 32, 64)	0
Conv2D	(None, 16, 16, 128)	131,072
BatchNormalization	(None, 16, 16, 128)	512
LeakyReLU	(None, 16, 16, 128)	0
Dropout	(None, 16, 16, 128)	0
Conv2D	(None, 8, 8, 256)	524,288
BatchNormalization	(None, 8, 8, 256)	1,024
LeakyReLU	(None, 8, 8, 256)	0
Dropout	(None, 8, 8, 256)	0
Conv2D	(None, 4, 4, 512)	2,097,152
BatchNormalization	(None, 4, 4, 512)	2,048
LeakyReLU	(None, 4, 4, 512)	0
Dropout	(None, 4, 4, 512)	0
Conv2D	(None, 1, 1, 1)	8,192
Flatten	(None, 1)	0

```
discriminator_input = layers.Input(shape=(64, 64, 1)) ❶
x = layers.Conv2D(64, kernel_size=4, strides=2, padding="same", use_bias = False)(
    discriminator_input
) ❷
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(
    128, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum = 0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(
    256, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum = 0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(
    512, kernel_size=4, strides=2, padding="same", use_bias = False
)(x)
x = layers.BatchNormalization(momentum = 0.9)(x)
x = layers.LeakyReLU(0.2)(x)
x = layers.Dropout(0.3)(x)
x = layers.Conv2D(
    1,
    kernel_size=4,
    strides=1,
    padding="valid",
    use_bias = False,
    activation = 'sigmoid'
)(x)
discriminator_output = layers.Flatten()(x) ❸
```

对抗学习优化目标

$$\min_G \max_D \mathbb{E}_{x \sim data} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]$$

最小最大过程



包含两个过程迭代进行： min_G 与 max_D

对抗学习优化

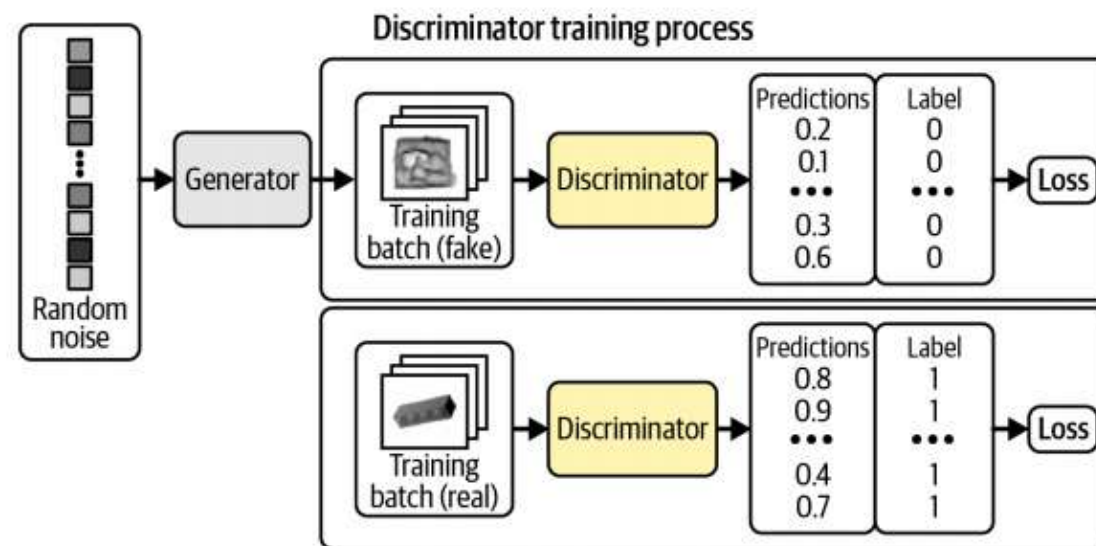
对抗学习优化—判别器优化

$$\max_D \mathbb{E}_{x \sim data} [\log D(x)] + \mathbb{E}_{x' \sim p_g} [\log(1 - D(x'))]$$

向1推 向0推

➤ D-Step: 固定 G , 优化 D

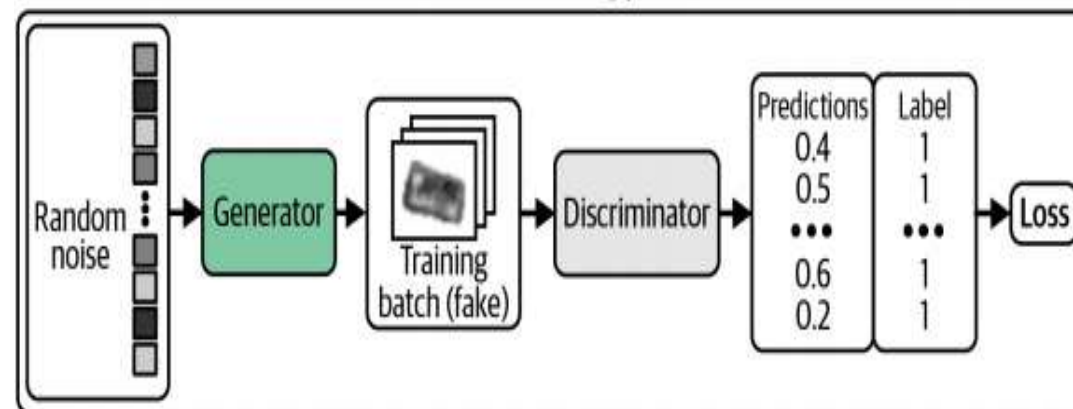
- D 用于区分真实样本与生成样本
- 二分类任务 (交叉熵损失)



对抗学习优化—生成器优化

$$\min_G \mathbb{E}_{x \sim data} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log (1 - D(G(z)))]$$

➤ G步骤：固定 D ，优化 G



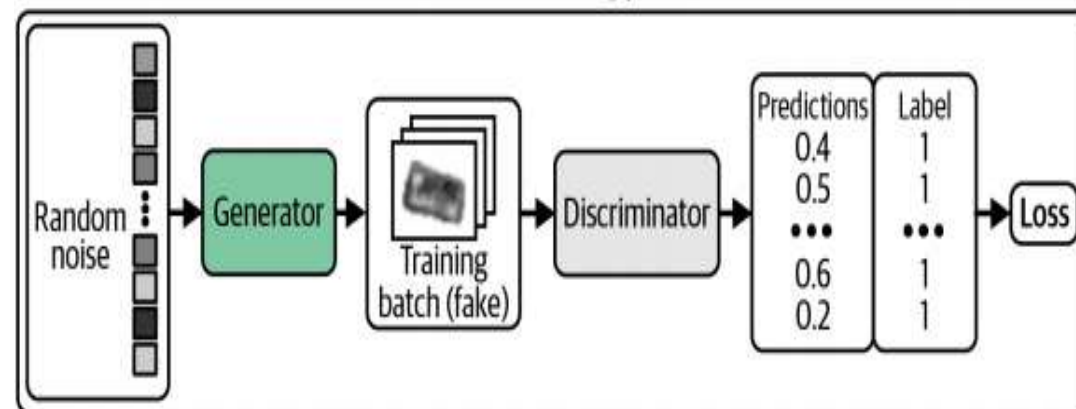
对抗学习优化—生成器优化

$$\min_G \mathbb{E}_{z \sim p_z} [\log(1 - \textcolor{red}{D(G(z))})]$$

推向1

➤ G步骤：固定 D ，优化 G

- 生成伪造数据，使得 D 将其判别为“真实”。
- G 的目标是“迷惑” D



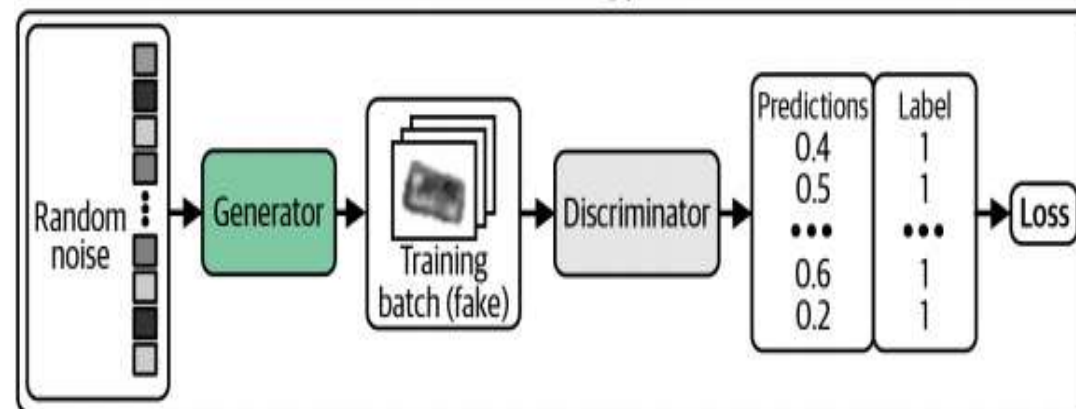
对抗学习优化—生成器优化

$$\max_G \mathbb{E}_{z \sim p_z} [\log D(G(z))]$$

推向1

➤ G步骤：固定 D ，优化 G

- 生成伪造数据，使得 D 将其判别为“真实”。
- G 的目标是“迷惑” D



对抗学习优化

对抗学习优化—D-G迭代优化流程

➤ K步判别器优化max迭代

➤ 1步生成器优化min 迭代

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{data}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

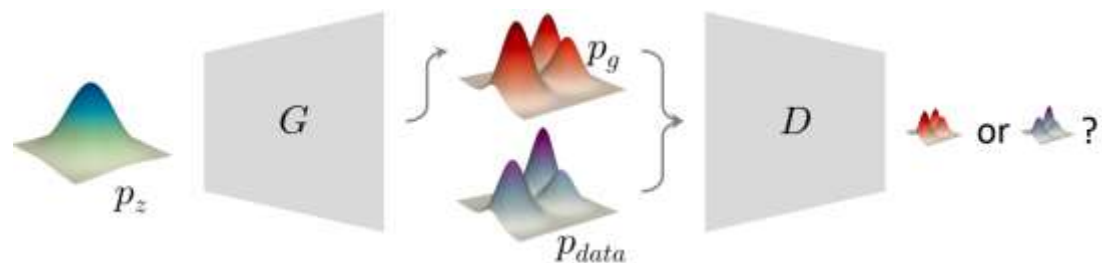
end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log D(G(z^{(i)})).$$

end for

The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.



对抗学习优化---D-G迭代优化流程

```
class DCGAN(models.Model):
    def __init__(self, discriminator, generator, latent_dim):
        super(DCGAN, self).__init__()
        self.discriminator = discriminator
        self.generator = generator
        self.latent_dim = latent_dim

    def compile(self, d_optimizer, g_optimizer):
        super(DCGAN, self).compile()
        self.loss_fn = losses.BinaryCrossentropy() ❶
        self.d_optimizer = d_optimizer
        self.g_optimizer = g_optimizer
        self.d_loss_metric = metrics.Mean(name="d_loss")
        self.g_loss_metric = metrics.Mean(name="g_loss")

    @property
    def metrics(self):
        return [self.d_loss_metric, self.g_loss_metric]
```

对抗学习优化



```
def train_step(self, real_images):  
    batch_size = tf.shape(real_images)[0]
```

number of training iterations **do**

k steps **do**

Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.

Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.

Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

for

Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.

Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log D(G(z^{(i)}))$$

or

Gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

```
self.loss_fn = losses.BinaryCrossentropy() 1
```

```

gradients_of_discriminator = disc_tape.gradient(
    d_loss, self.discriminator.trainable_variables
)
gradients_of_generator = gen_tape.gradient(
    g_loss, self.generator.trainable_variables
)

self.d_optimizer.apply_gradients(
    zip(gradients_of_discriminator, discriminator.trainable_variables)
) ⑧
self.g_optimizer.apply_gradients(
    zip(gradients_of_generator, generator.trainable_variables)
)

self.d_loss_metric.update_state(d_loss)
self.g_loss_metric.update_state(g_loss)

return {m.name: m.result() for m in self.metrics}
    
```

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{\text{data}}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(x^{(i)}) + \log (1 - D(G(z^{(i)}))) \right].$$

id for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log D(G(z^{(i)}))$$

for

gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

对抗学习优化---训练过程

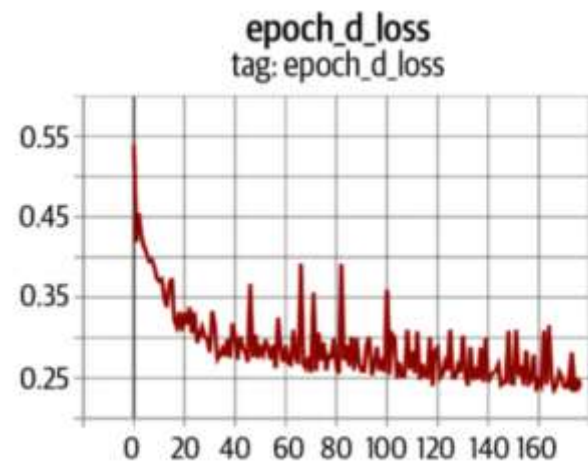
```
dcgan = DCGAN(
    discriminator=discriminator, generator=generator, latent_dim=100
)

dcgan.compile(
    d_optimizer=optimizers.Adam(
        learning_rate=0.0002, beta_1 = 0.5, beta_2 = 0.999
    ),
    g_optimizer=optimizers.Adam(
        learning_rate=0.0002, beta_1 = 0.5, beta_2 = 0.999
    ),
)

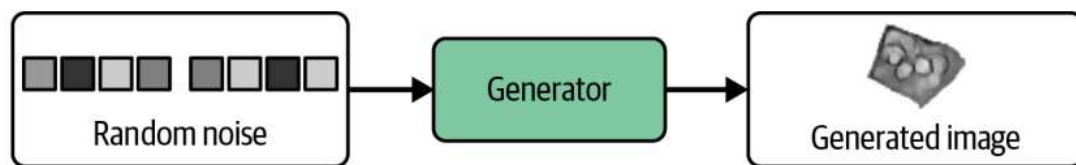
dcgan.fit(train, epochs=300)
```

$$\min_G \mathbb{E}_{z \sim p_z} [-\log(D(G(z)))]$$

$$\min_D \mathbb{E}_{x \sim data} [-\log D(x)] + \mathbb{E}_{z \sim p_z} [-\log (1 - D(G(z)))]$$

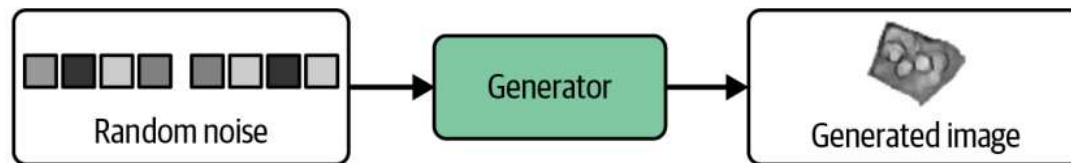


不同训练轮次下的生成采样

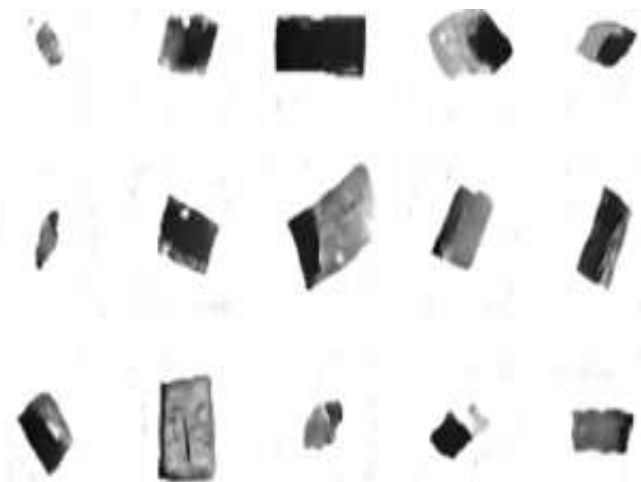


生成的图像真实吗?

GAN图像生成的质量



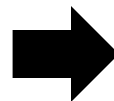
Generated images



Ground true

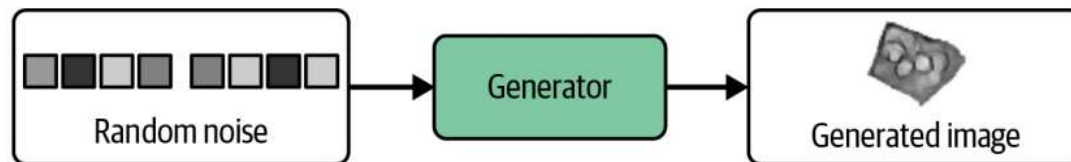


$L1$ 距离下最近的样本



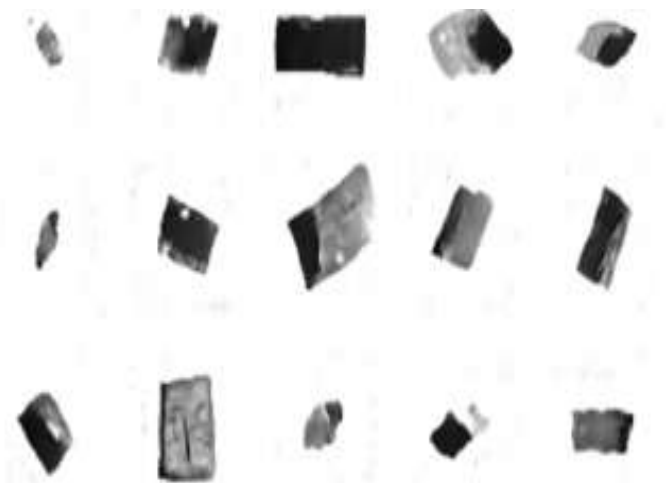
生成样本很像数据集中样本，但也有些不同。

GAN图像生成的质量



Generated images

GAN



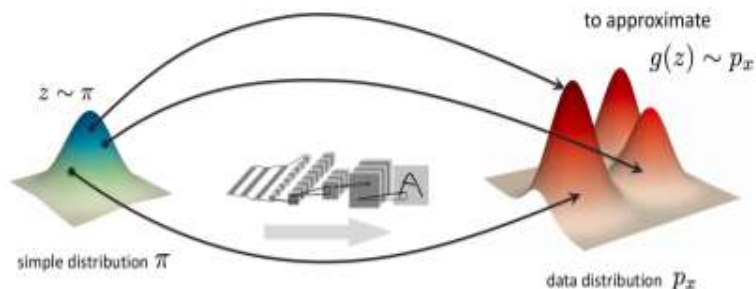
VAE



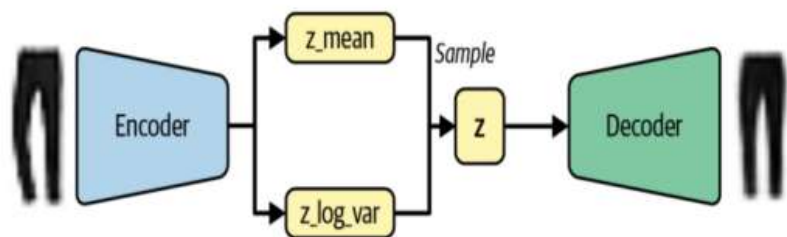
相比VAE，GAN生成图像边界更加清晰

GAN符合深度生成模型统一范式吗？

“简单分布采样 + 复杂映射 = 复杂分布采样”

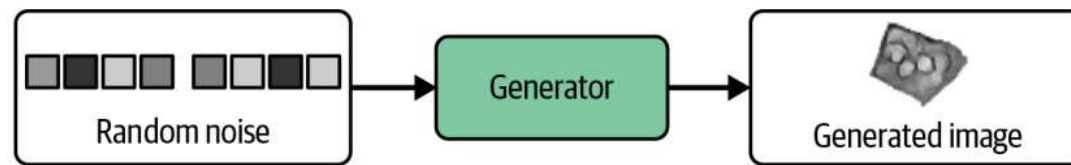


变分自编码器



1. 自编码器：隐变量 z 和样本 x 之间的复杂映射。
2. 变分：确保 z 为简单高斯分布。

对抗神经网络

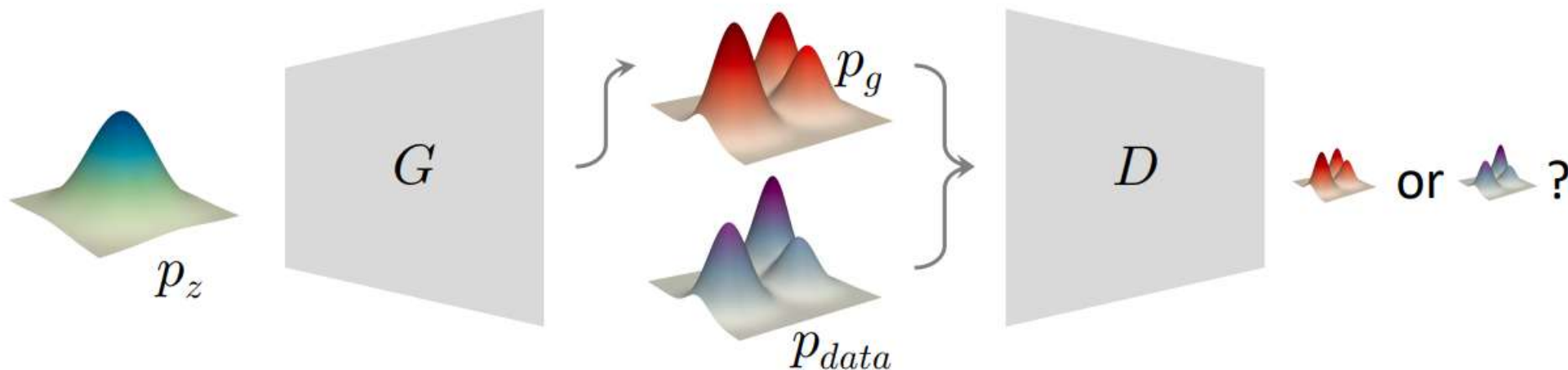


1. 指定 z 采样自简单高斯分布
2. 生成器构建隐变量 z 到样本 x 的复杂映射

第二节 理论探索： 如何理解对抗学习优化

GAN优化目标： min-max迭代优化
生成式模型目标： 建模真实数据分布

$$\min_G \max_D V(G, D) = E_{x \sim data} [\log D(x)] + E_{z \sim p_z} [\log (1 - D(G(z)))]$$

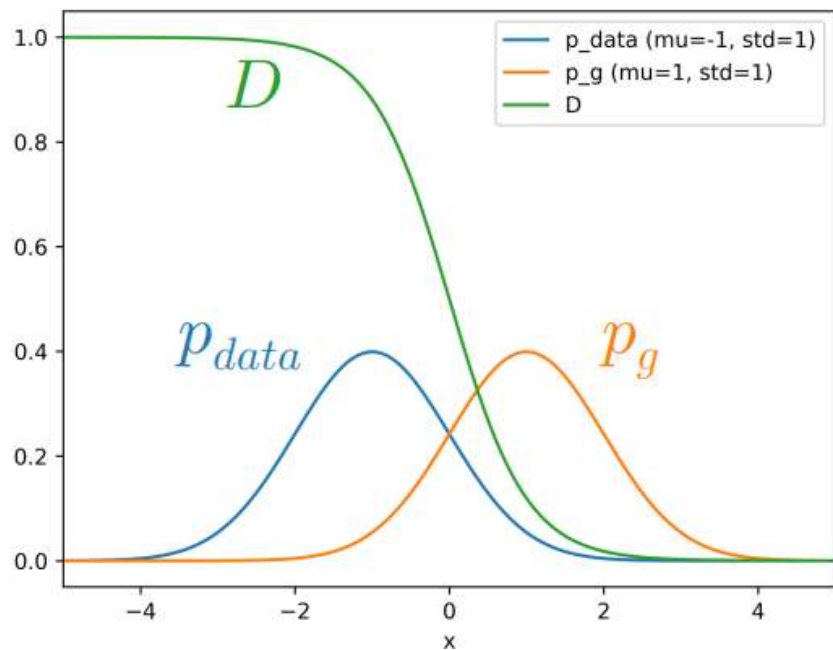


为什么min-max迭代优化过程可以建模数据分布？

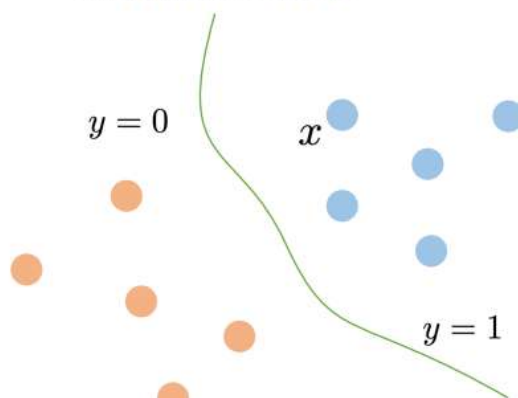
理论探索： 判别器最优解

$$\max_D E_{x \sim p_{data}} [\log D(x)] + E_{x \sim p_G} [\log(1 - D(x))] \quad \text{生成器 } P_g \text{ 固定}$$

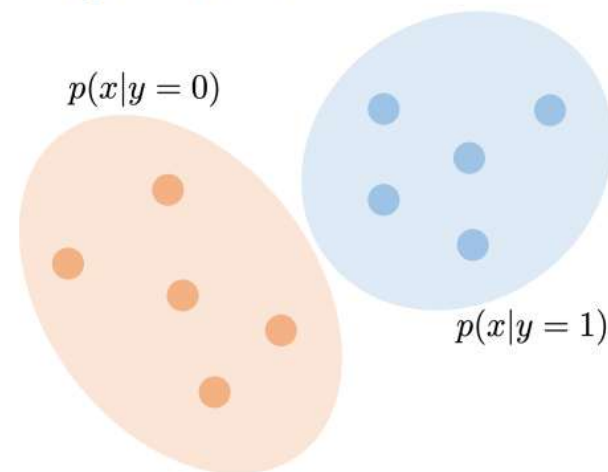
$$D_G^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$$



discriminative



generative



理论探索：最优判别器 D^* 下，对抗优化目标

$$\min_G V(G, D) = E_{x \sim p_{data}} [\log D(x)] + E_{x \sim p_G} [\log(1 - D(x))]$$

↓ 代入最优判别器

$$\min_G V(G, D_G^*) = E_{x \sim p_{data}} \left[\log \frac{p_{data}(x)}{p_{data}(x) + p_G(x)} \right] + E_{x \sim p_G} \left[\log \left(\frac{p_G(x)}{p_{data}(x) + p_G(x)} \right) \right]$$

$$D_{KL}(P||Q) = \int P(x) \log \frac{P(x)}{Q(x)} = E_{x \sim P} [\log \frac{P(x)}{Q(x)}]$$

$$\min_G V(G, D_G^*) = E_{x \sim p_{data}} \left[\log \frac{p_{data}(x)}{\frac{p_{data}(x) + p_G(x)}{2}} \right] + E_{x \sim p_G} \left[\log \left(\frac{p_G(x)}{\frac{p_{data}(x) + p_G(x)}{2}} \right) \right] - 2 \log 2$$

$$\min_G V(G, D_G^*) = D_{KL} \left[p_{data}, \frac{p_{data} + p_G}{2} \right] + D_{KL} \left[p_G, \frac{p_{data} + p_G}{2} \right] - 2 \log 2$$

$2D_{JS}[p_{data}, p_G]$ (JSD散度)

理论探索：最有判别器 D^* 下，对抗优化目标

Jensen-Shannon(JSD)散度

$$\min_G V(G, D_G^*) = D_{KL} \left[p_{data}, \frac{p_{data} + p_G}{2} \right] + D_{KL} \left[p_G, \frac{p_{data} + p_G}{2} \right] - 2\log 2$$

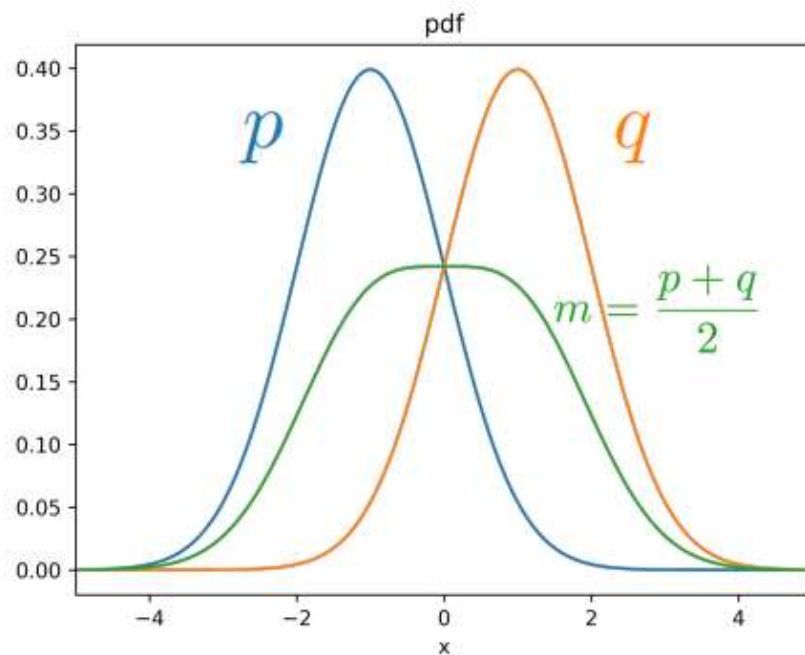
$2D_{JS}[p_{data}, p_G]$ (JSD散度)



$$D_{JS}[p, q] = \frac{1}{2} (D_{KL}[p, \frac{p+q}{2}] + D_{KL}[q, \frac{p+q}{2}])$$

背景：Jensen-Shannon 散度

- $D_{JS}(p||q) \triangleq \frac{1}{2} D_{KL}(p||\frac{p+q}{2}) + \frac{1}{2} D_{KL}(q||\frac{p+q}{2})$
- D_{JS} : “p和q到平均值的KL散度均值”



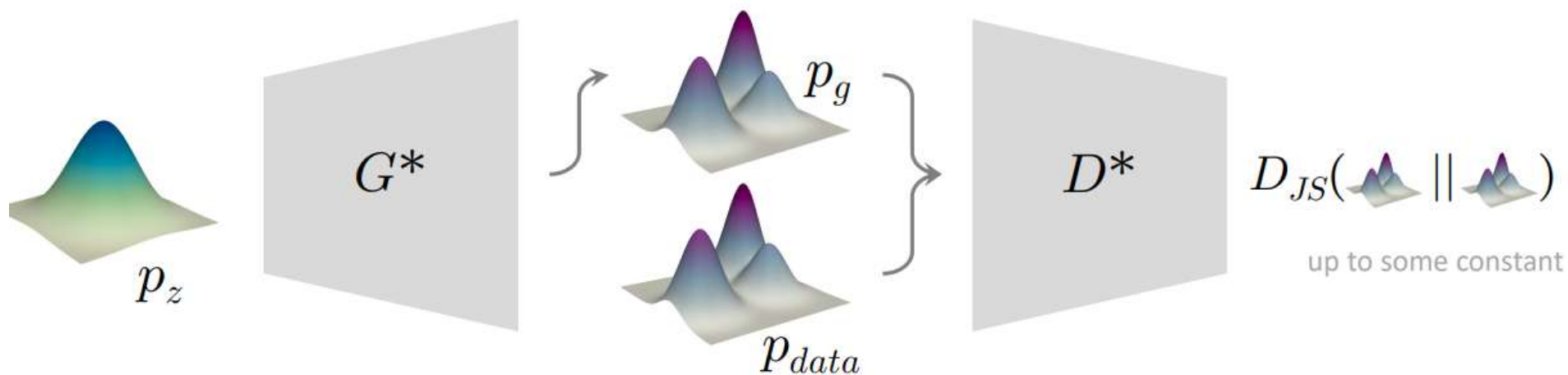
背景：Jensen–Shannon 散度

- $D_{JS}(p||q) \triangleq \frac{1}{2} D_{KL}(p||\frac{p+q}{2}) + \frac{1}{2} D_{KL}(q||\frac{p+q}{2})$
- D_{JS} : “p和q到平均值的KL散度均值”
- 特点:
 - D_{JS} 对称;而 D_{KL} 不对称
 - D_{JS} 有界: $[0, \log 2]$;而 D_{KL} 无界: $[0, \infty)$
 - 当 $p=q$ 时最优

理论探索：全局最优

➤ 当 $p_g = p_{data}$ 时达到全局最优

$$\min_G V(G, D^*) = 2D_{JS}(p_{data} || p_g) - 2\log 2$$



总结

- 对于任意给定的 G ，其最优的判别器 D 为：

$$D_G^* = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$$

- 在最优判别器 D_G 下，GAN 优化的是 Jensen–Shannon 散度：

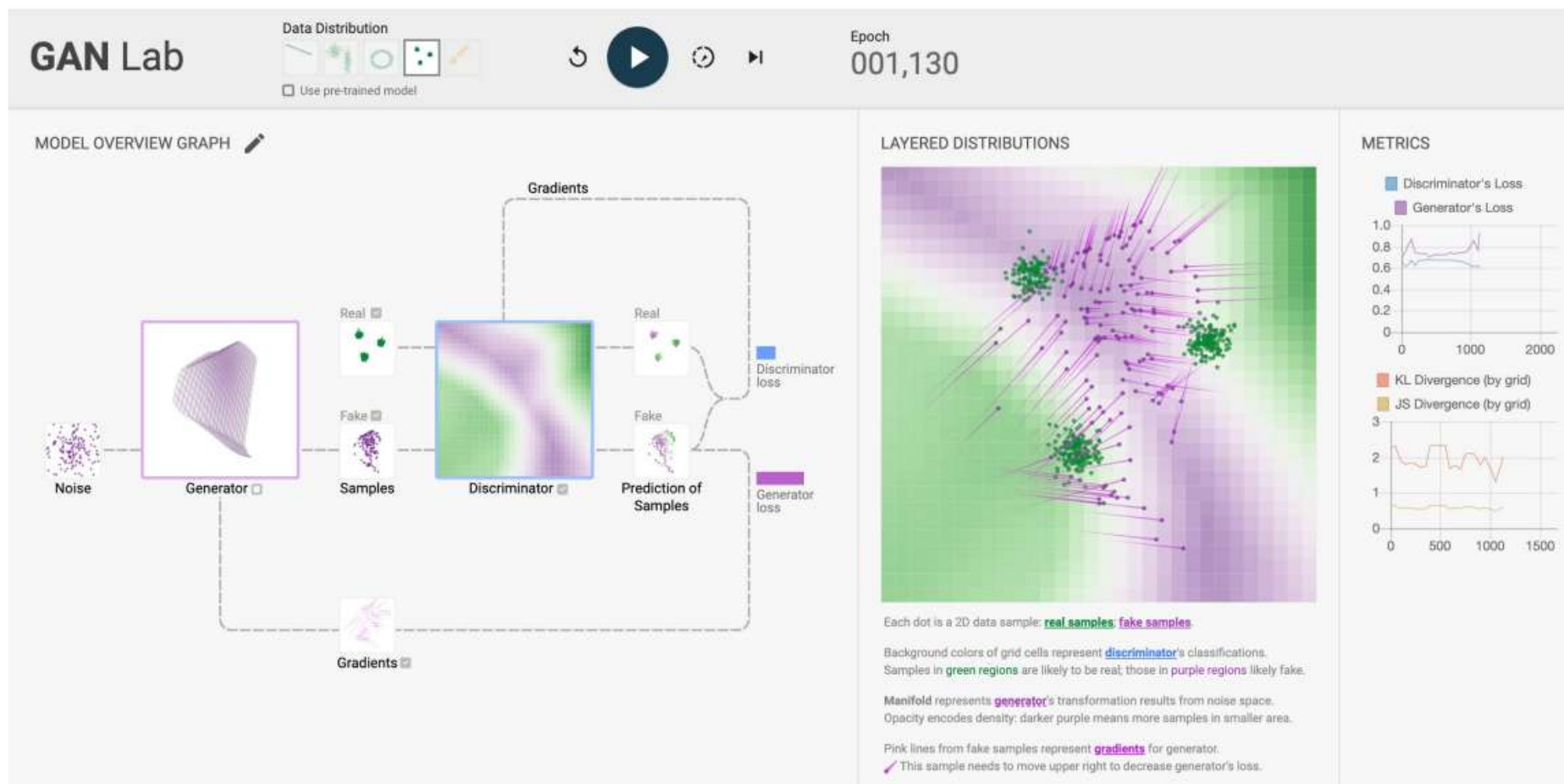
$$\min_G \mathcal{L}(D^*, G) = 2D_{JS}(p_{data}(x) || p_g(x)) - 2\log 2$$

- 当 $p_g = p_{data}$ 时达到全局最优：

$$\mathcal{L}(D^*, G^*) = -2\log 2$$

执行样例：GAN LAB

<https://poloclub.github.io/ganlab/>



作业

- 作业：基于图像数据集(bricks, faces, or flowers)构建 DCGAN
- 参考代码见github主页
- 要求：
 - 基于pytorch编写代码
 - 提交运行完整的jupyter

网盘提交：

<https://pan.nuaa.edu.cn/collection/bf031c4ca430e048cdb024e741aa71fe>

文件命名规则：**homework3-姓名-学号.ipynb**

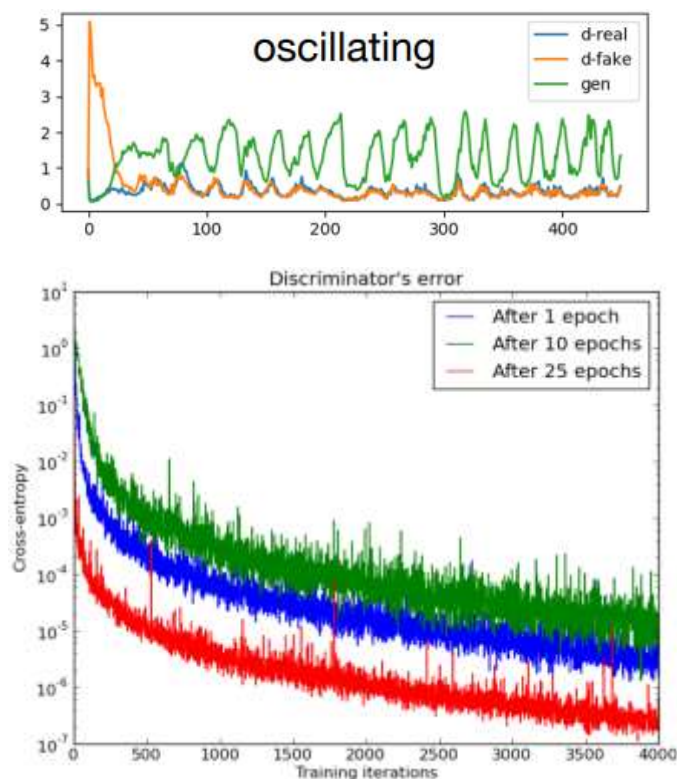
截止时间：**下周周日**



第三节 WGAN

GAN的问题

- 难以训练 / 收敛
 - 难以达到均衡
 - 梯度消失
 - 模式崩塌



W-GAN简要介绍

➤ 对数学家来说:

- 使用 Wasserstein 距离, 而非 JS 散度

➤ 对工程师来说:

- 移除对数运算
- 裁剪权重

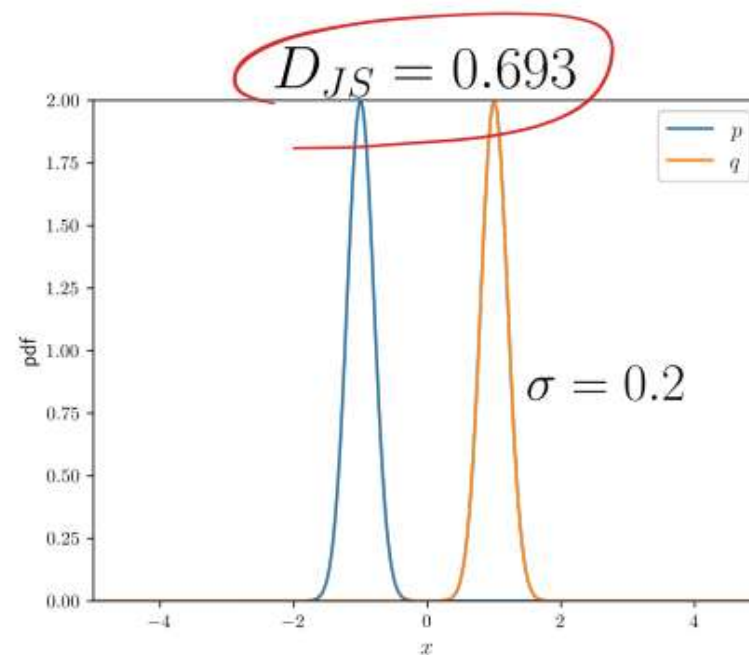
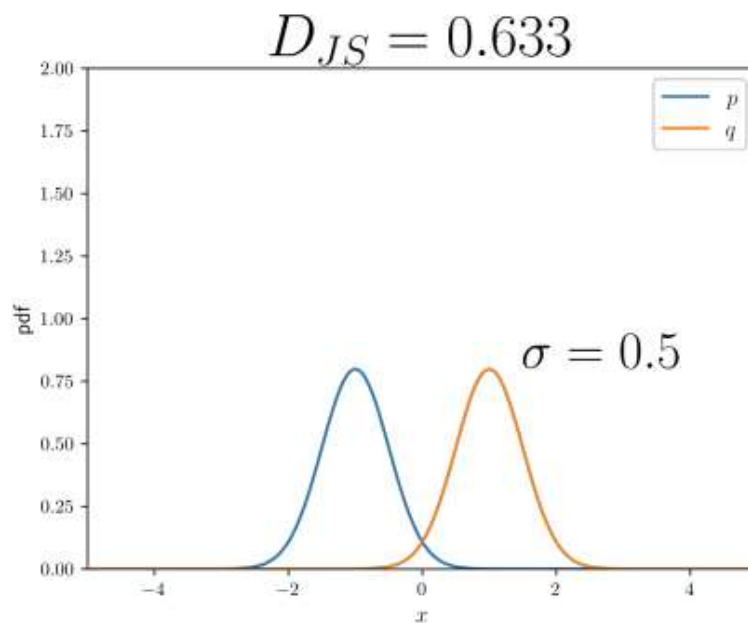
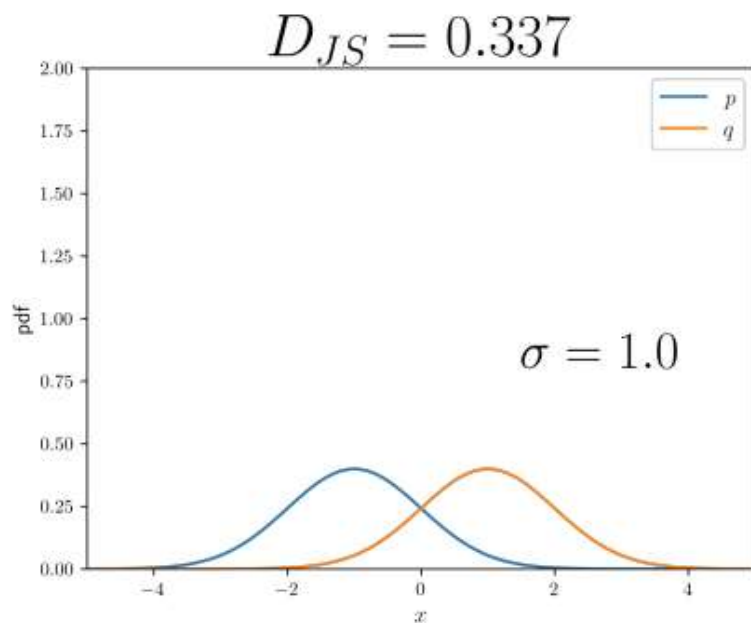
$$\min_G \mathcal{L}(D^*, G) = 2D_{JS}(p_{data}(x) || p_g(x)) - 2\log 2$$



$$\min_G \mathcal{L}(D^*, G) = W(p_{data}(z), p_g(x))$$

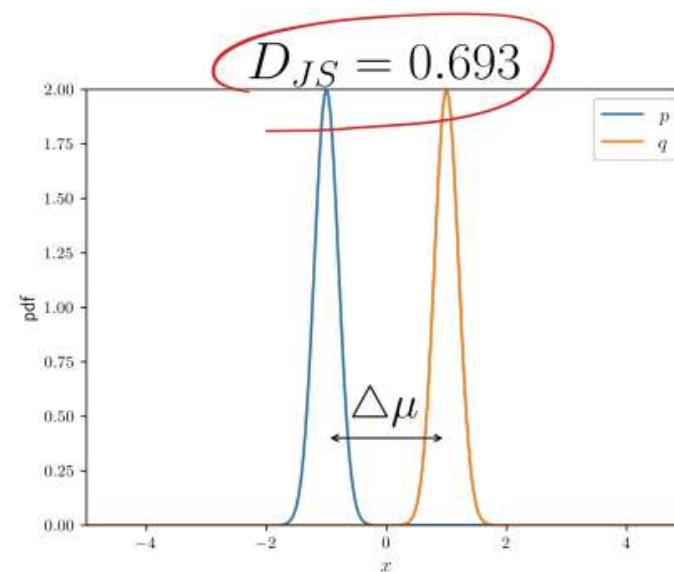
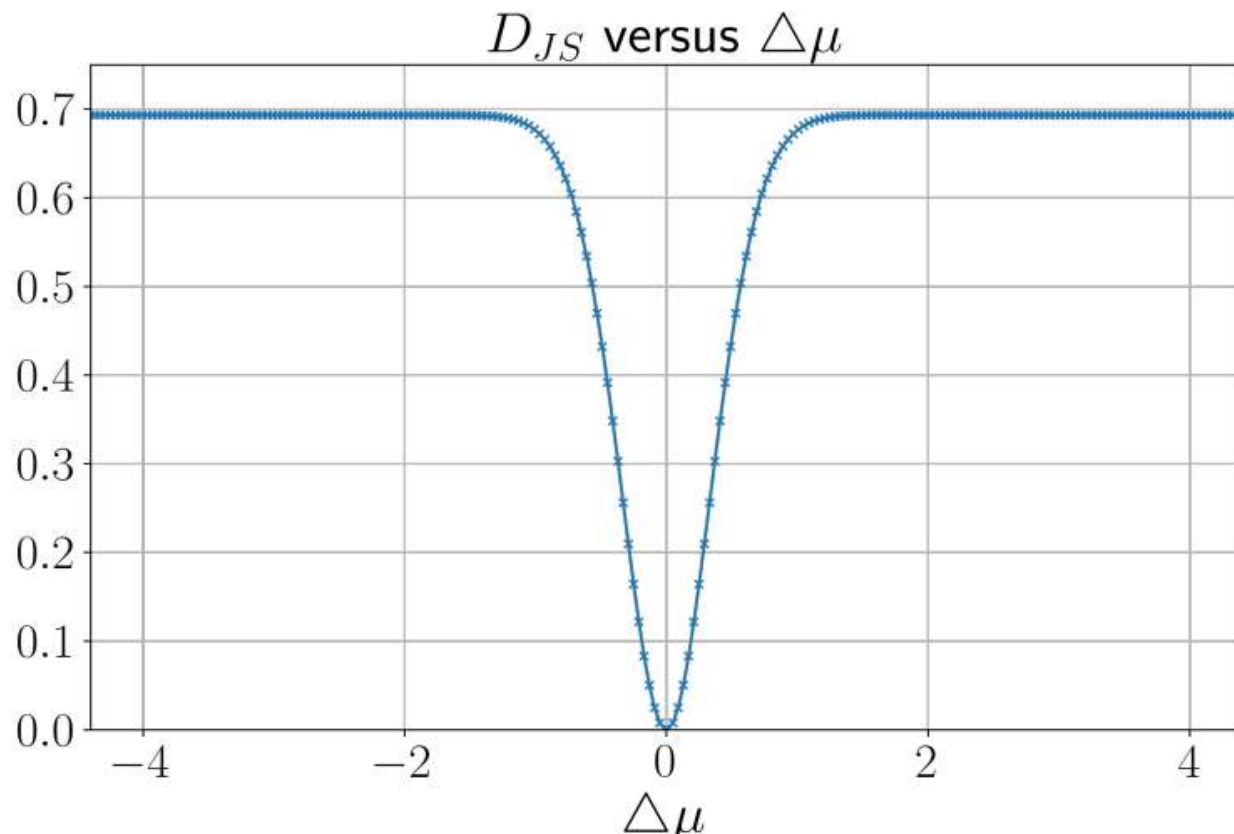
D_{JS} 的问题

如果 p 和 q 没有重叠, D_{JS} 是一个常数 (log2上界)。



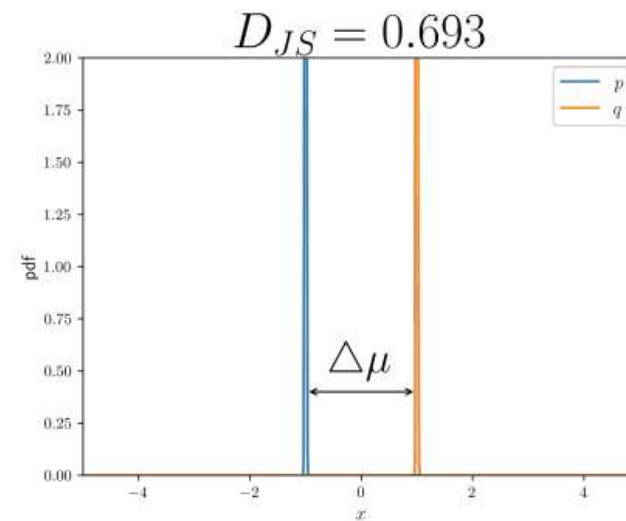
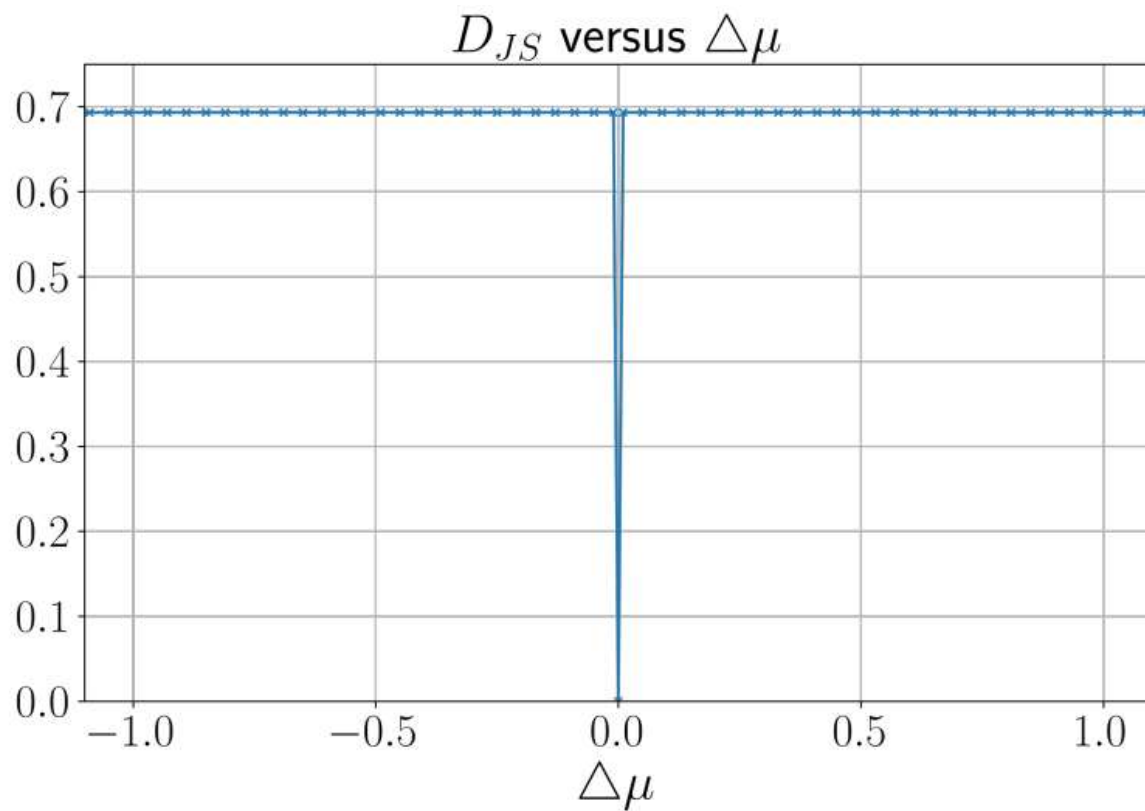
D_{JS} 的问题

D_{JS} 仅仅当 p 和 q 非常接近的时候才有用



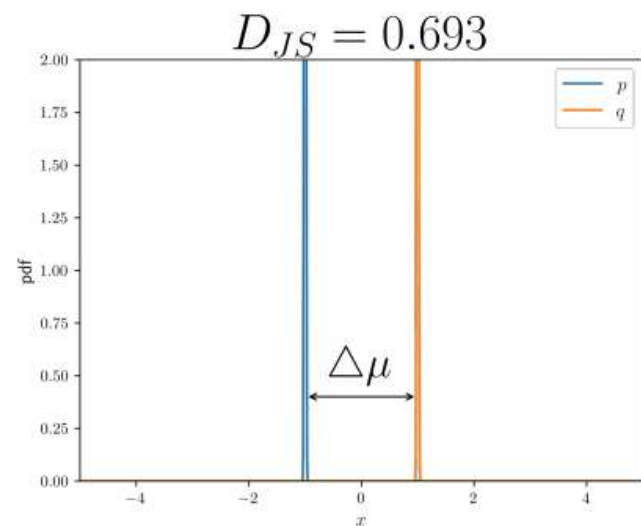
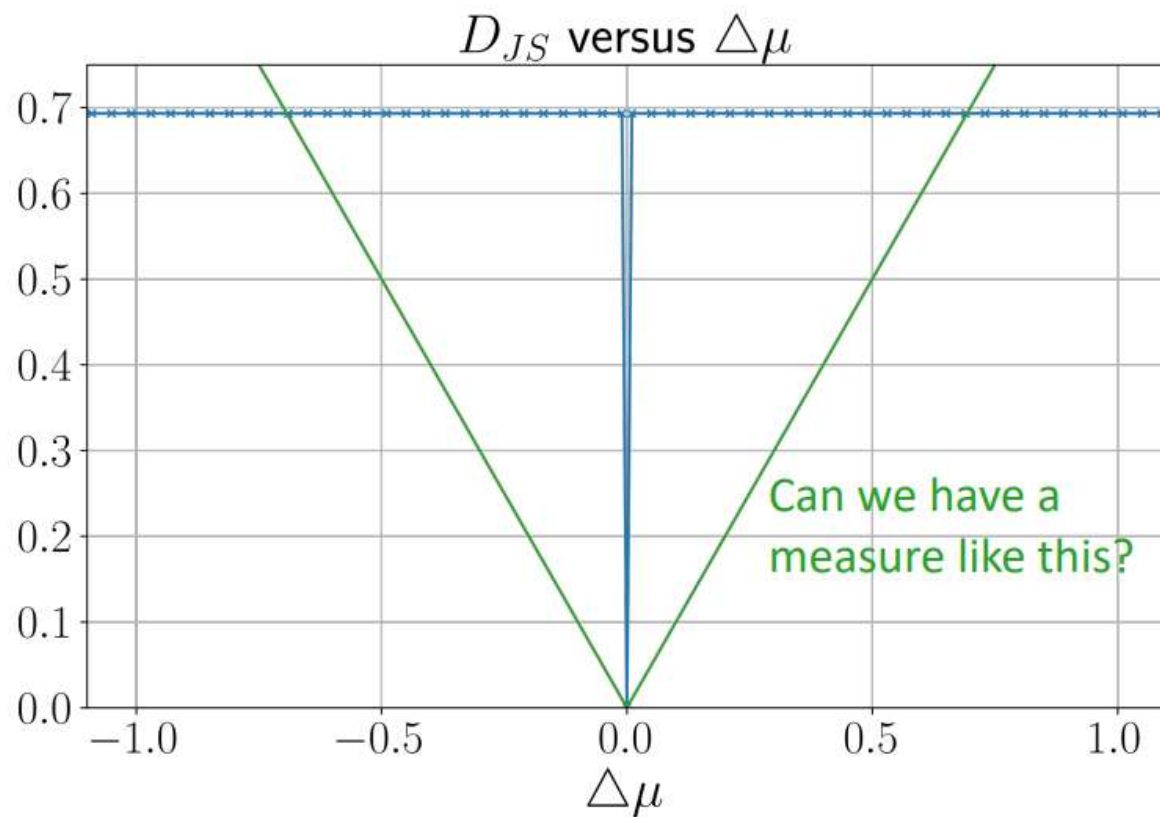
D_{JS} 的问题

当 p 和 q 都是 delta 函数时, D_{JS} 也是一个 delta 函数。



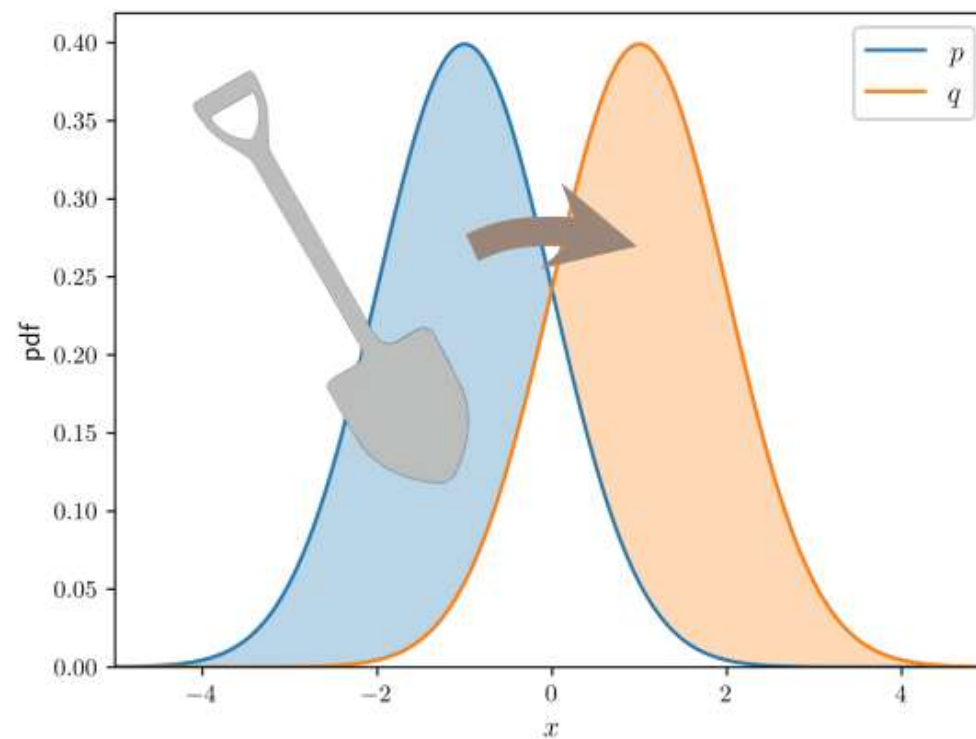
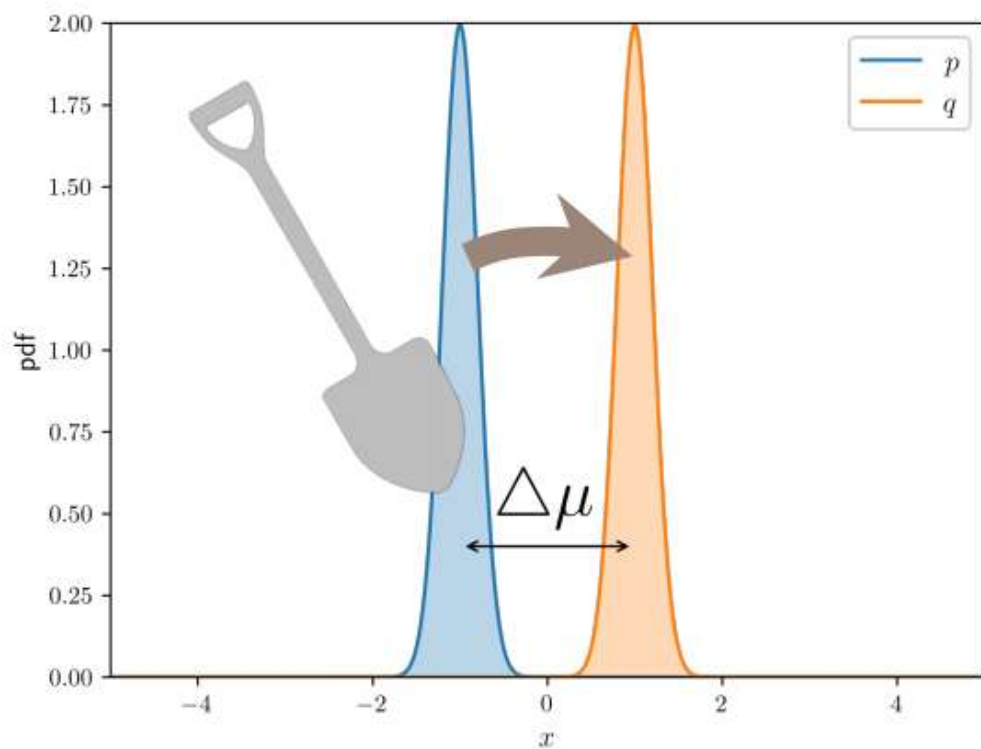
D_{JS} 的问题

当 p 和 q 都是 delta 函数时, D_{JS} 也是一个 delta 函数。

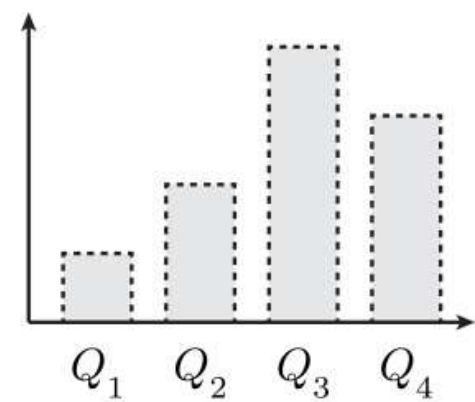
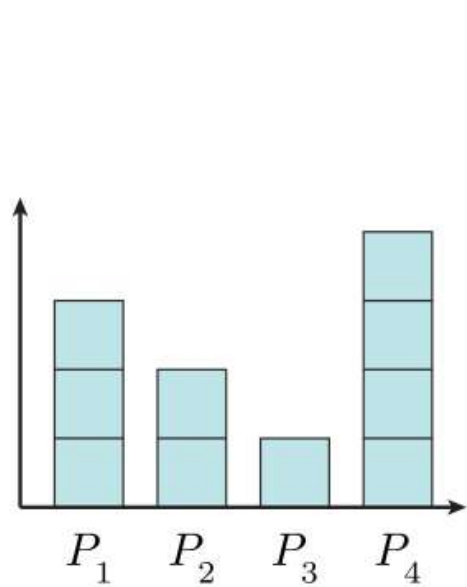


Wasserstein距离

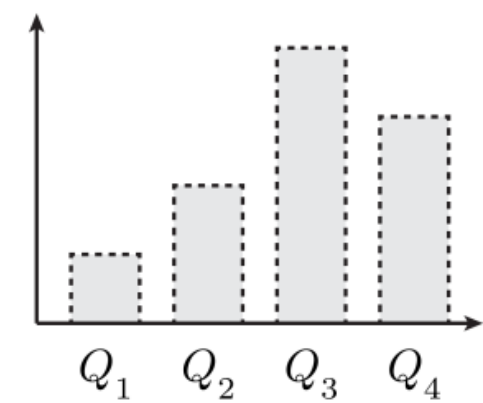
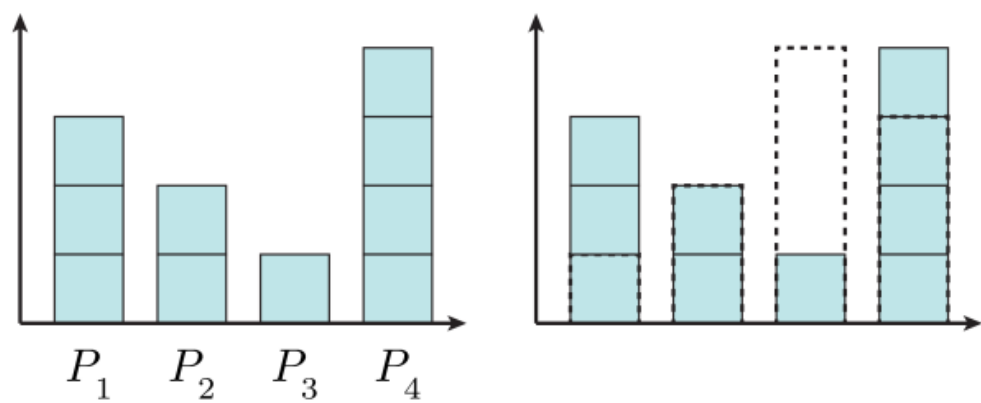
别名：推土机距离(Earth Mover's Distance)



Wasserstein距离



Wasserstein距离

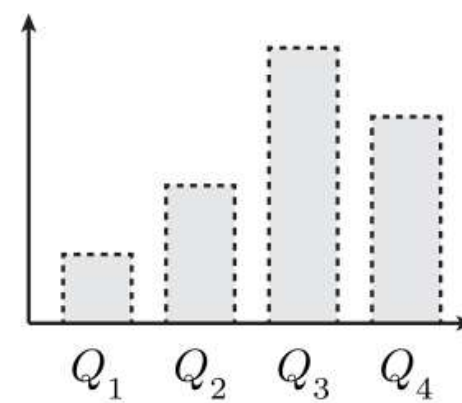
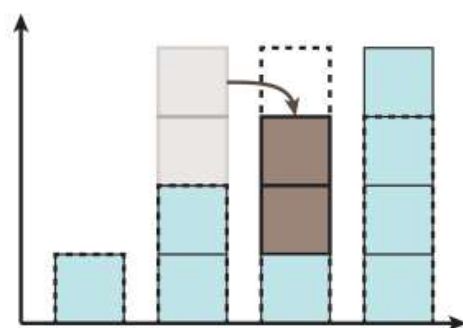
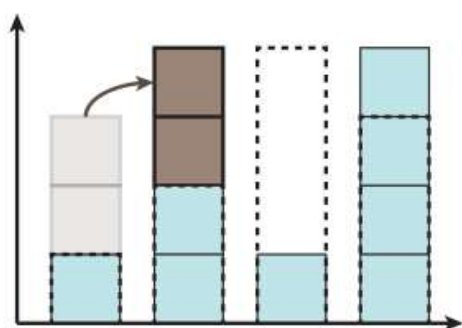
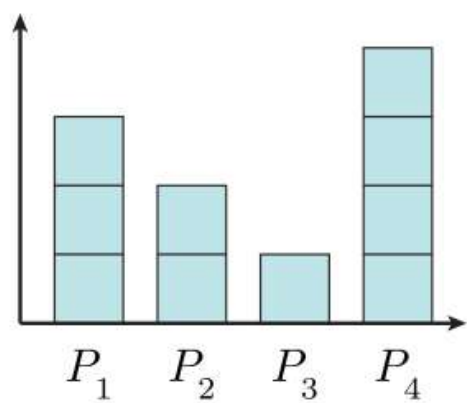


Wasserstein距离

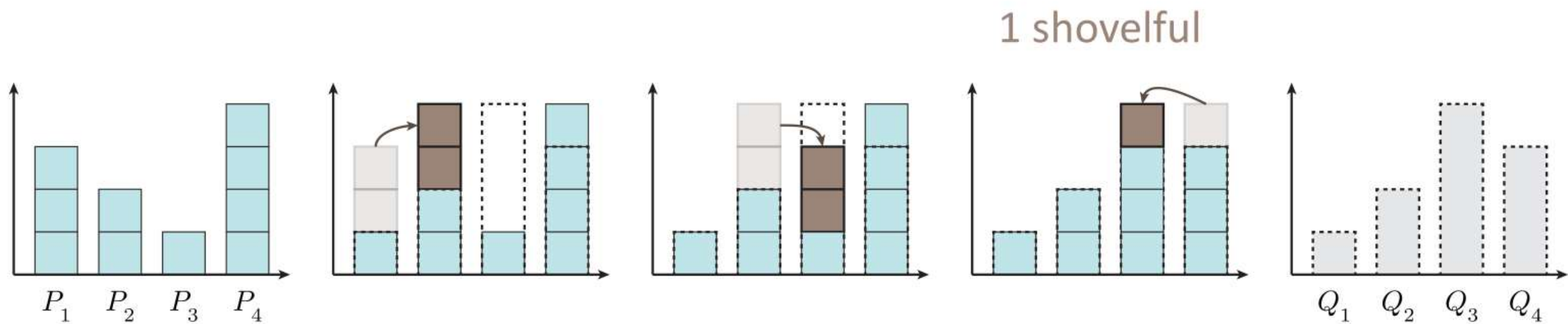


Wasserstein距离

2 shovelfuls

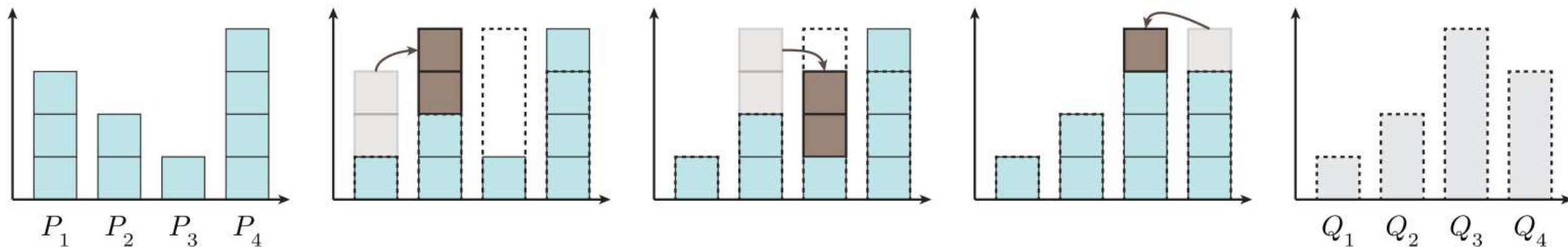


Wasserstein距离

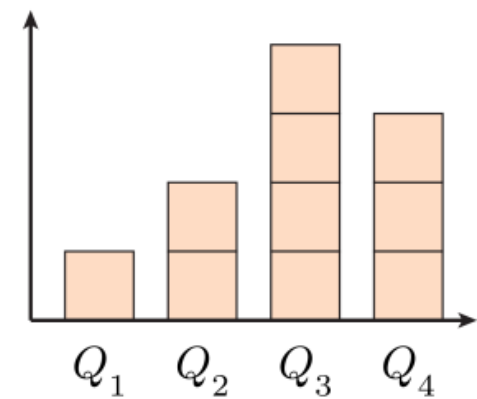
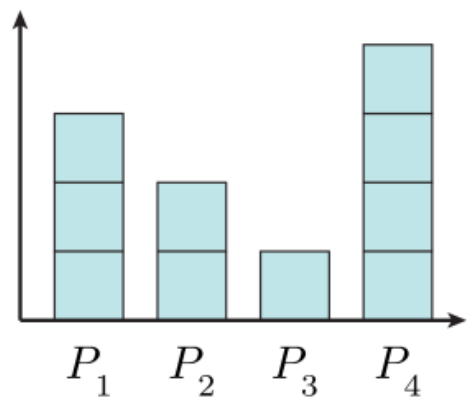


Wasserstein距离

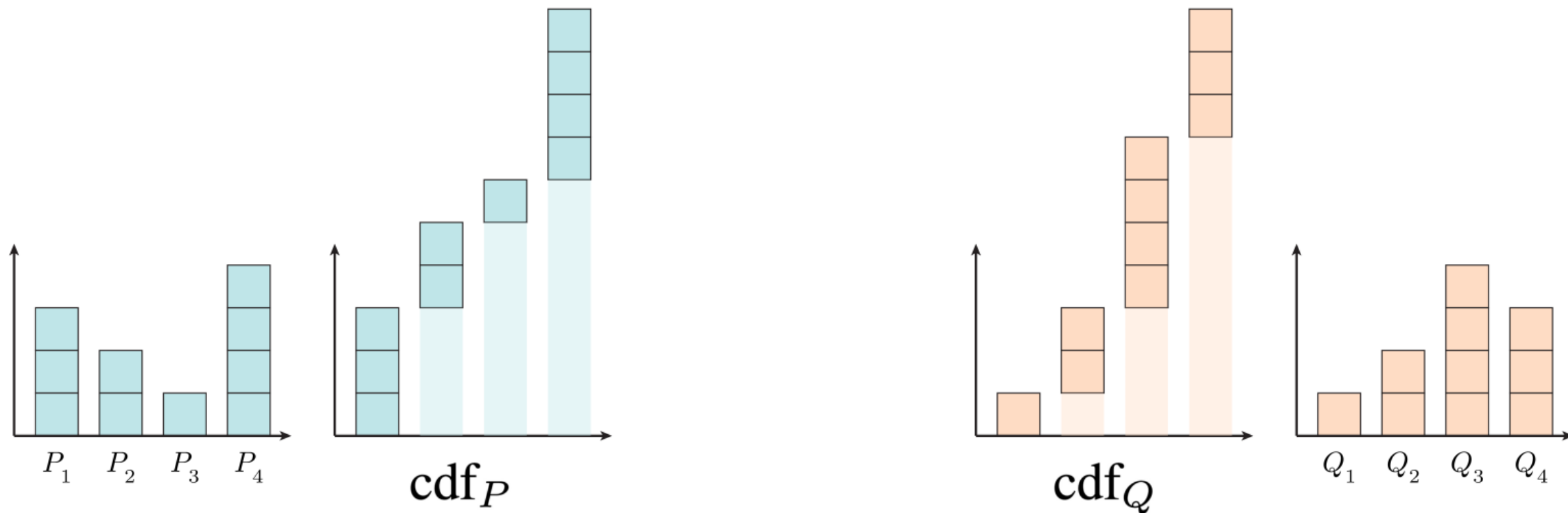
$$W(P, Q) = 5 \times \text{■}$$



Wasserstein距离

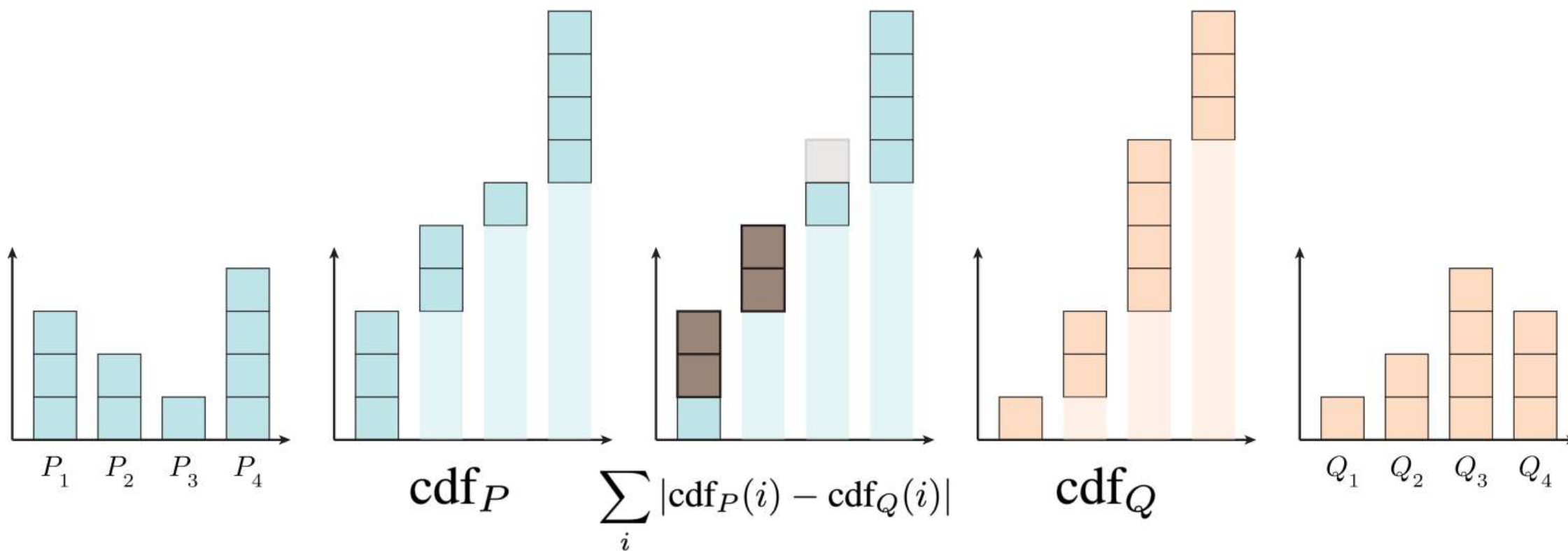


Wasserstein距离



cdf: 累积分布函数

Wasserstein距离

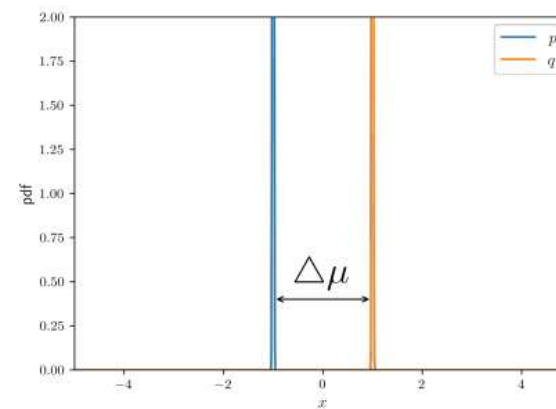
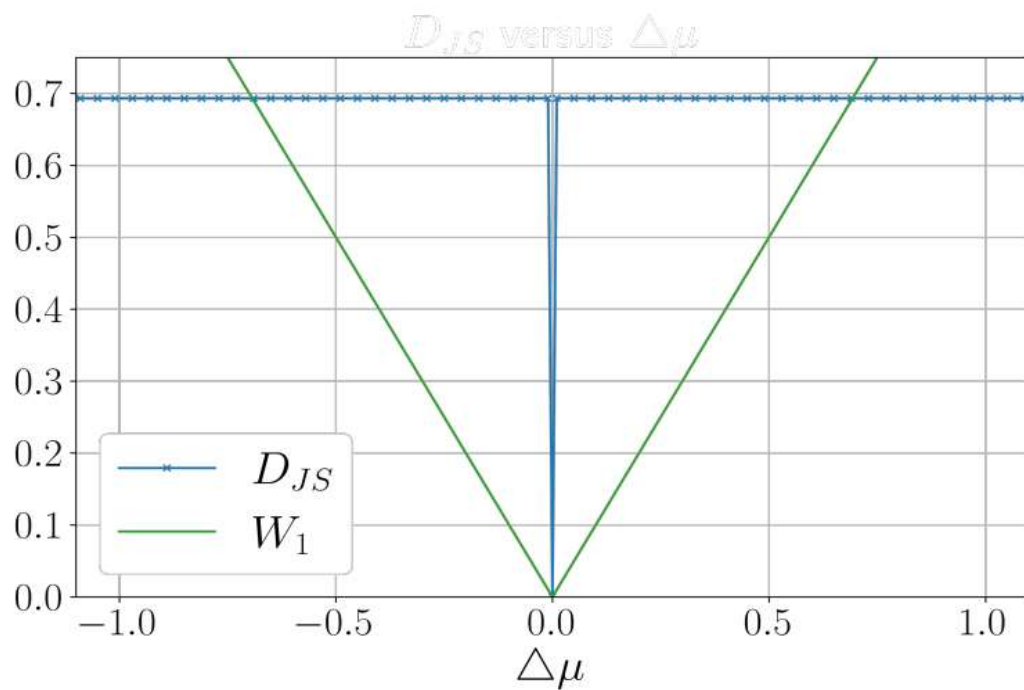


$$W(P, Q) = 5 \times \text{brown square}$$

Wasserstein距离

➤ 当 p 和 q 是delta公式:

$$W_1(p, q) = |\mu_p - \mu_q|$$



Wasserstein距离

➤ 一维离散分布的 1-Wasserstein 距离:

$$W_1(P, Q) = \sum_i |cdf_P(i) - cdf_Q(i)|$$

➤ 一维连续分布的 1-Wasserstein 距离:

$$W_1(p, q) = \int |cdf_p(x) - cdf_q(x)| dx$$

W-GAN 优化 Wasserstein 距离

- Kantorovich-Rubinstein 对偶公式:

$$W_1(p, q) = \sup_{\|f\| \leq 1} \mathbb{E}_{x \sim p}[f(x)] - \mathbb{E}_{x \sim q}[f(x)]$$

- K -Lipschitz 连续性:

所有1-Lipschitz函数

$$|f(x) - f(y)| \leq K|x - y|, \forall x, y$$

- 梯度有界:

$$\frac{|f(x) - f(y)|}{|x - y|} \leq K$$

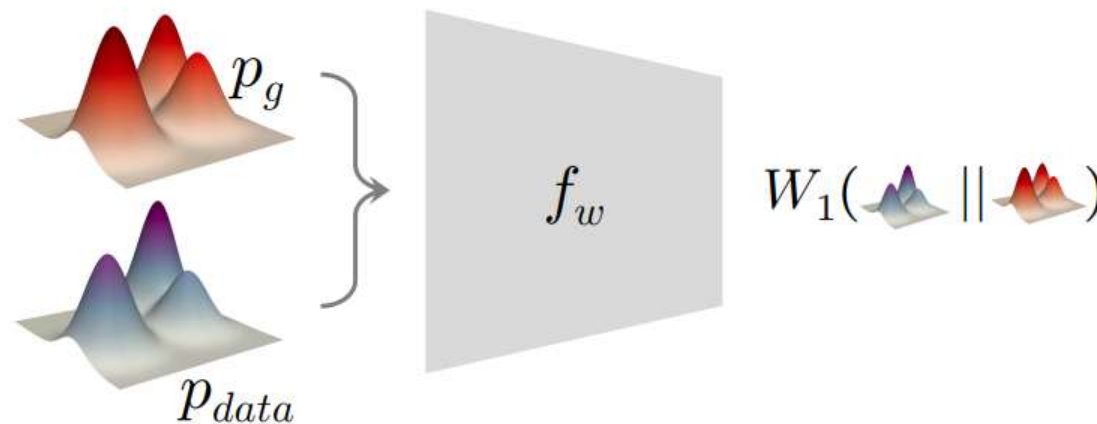
W-GAN 优化 Wasserstein 距离

➤ W-GAN的目标函数:

$$\min_G W_1(p_{data}, p_g) = \sup_{\|f\| \leq 1} \mathbb{E}_{x \sim p_{data}}[f(x)] - \mathbb{E}_{x \sim p_g}[f(x)]$$

$$\min_G \max_{f_w} \mathbb{E}_{x \sim p_{data}}[f_w(x)] - \mathbb{E}_{x \sim p_g}[f_w(x)]$$

权重有界：实践中通常裁剪至 $[-0.01, 0.01]$ 区间



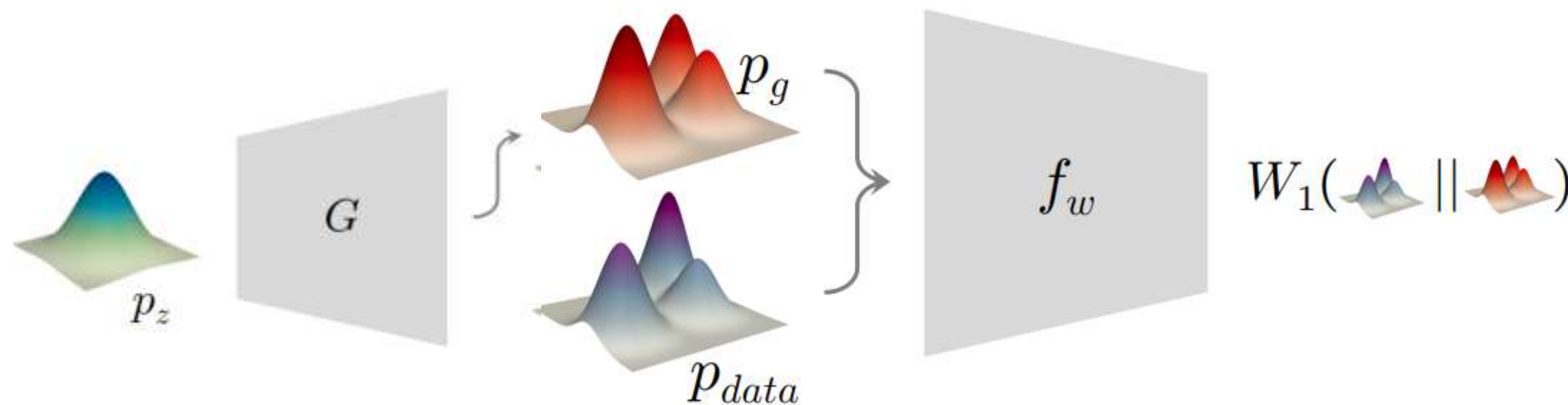
W-GAN 优化 Wasserstein 距离

➤ W-GAN的目标函数:

$$\min_G W_1(p_{data}, p_g) = \sup_{\|f\| \leq 1} \mathbb{E}_{x \sim p_{data}}[f(x)] - \mathbb{E}_{x \sim p_g}[f(x)]$$

$$\min_G \max_{f_w} \mathbb{E}_{x \sim p_{data}}[f_w(x)] - \mathbb{E}_{z \sim p_z}[f_w(G(z))]$$

权重有界：实践中通常裁剪至 $[-0.01, 0.01]$ 区间



W-GAN vs 原始GAN

- W-GAN的目标函数 (D步骤) :

$$\max_{w \in \mathcal{W}} \mathbb{E}_{x \sim p_{data}} [f_w(x)] - \mathbb{E}_{z \sim p_z} [f_w(G(z))]$$

- 原始GAN的目标函数 (D步骤) :

$$\max_D \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

W-GAN vs 原始GAN

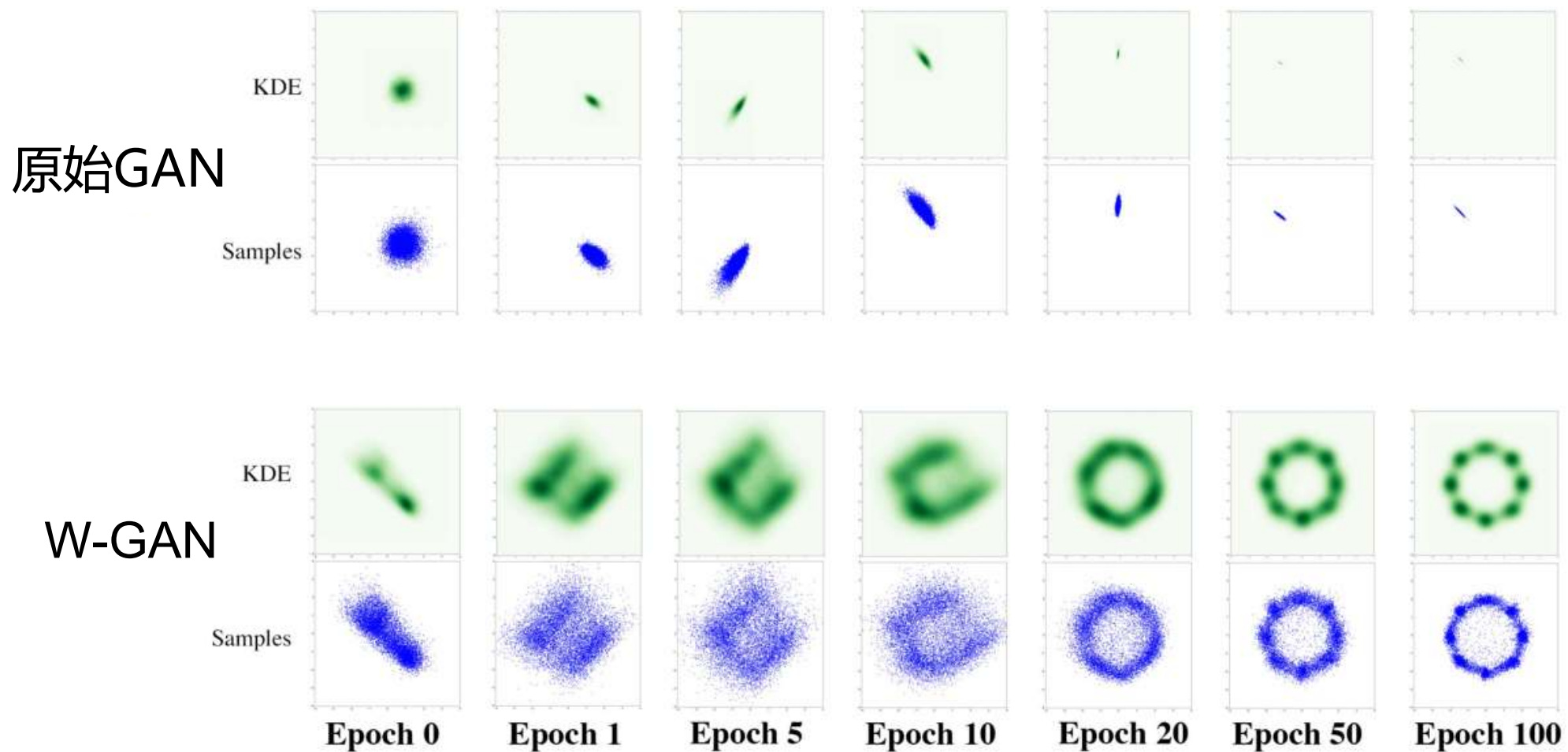
- W-GAN的目标函数（生成器）：

$$\min_G - \mathbb{E}_{z \sim p_z} [f_w(G(z))]$$

- 原始GAN的目标函数（生成器）：

$$\min_G \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

W-GAN vs 原始GAN



总结

- **核心思想:**GAN 由生成器与判别器构成, 通过对抗学习优化实现从随机噪声到逼真数据的生成, 本质是让生成器 “伪造” 数据以骗过判别器。
- **理论本质:**GAN 的优化目标等价于最小化真实分布与生成分布之间的 Jensen-Shannon 散度。其训练不稳定、梯度消失等问题源于 JSD 在分布不重叠时失去梯度。
- **关键改进:**WGAN 改用 Wasserstein 距离衡量分布差异, 通过 Lipschitz 约束 (如权重裁剪) 确保训练过程梯度平稳, 显著提升训练稳定性和生成质量。